Model-based testing

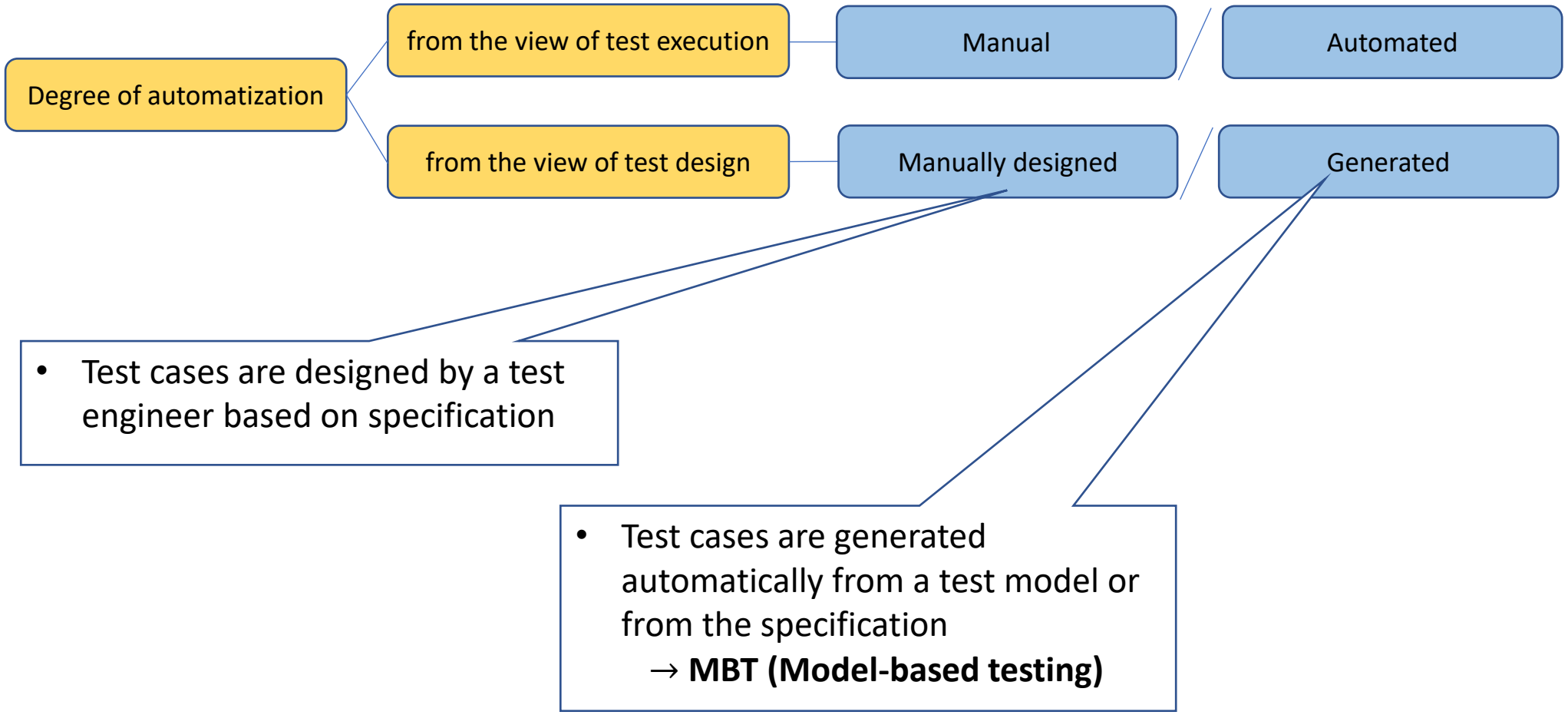
Gábor Árpád Németh

Version: 2024.03.19.4

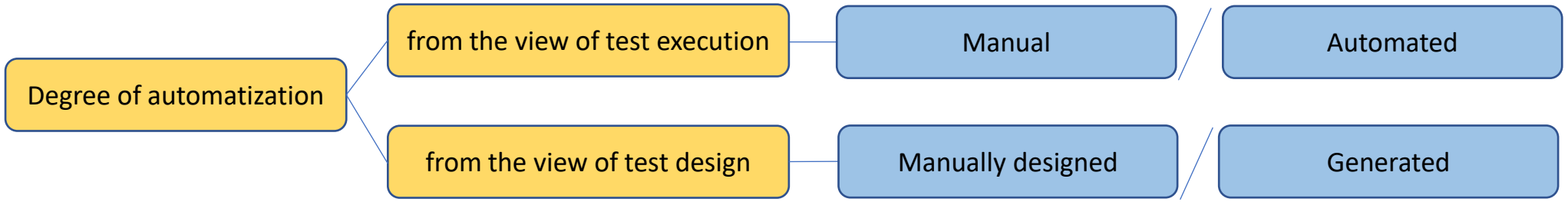
Part 1

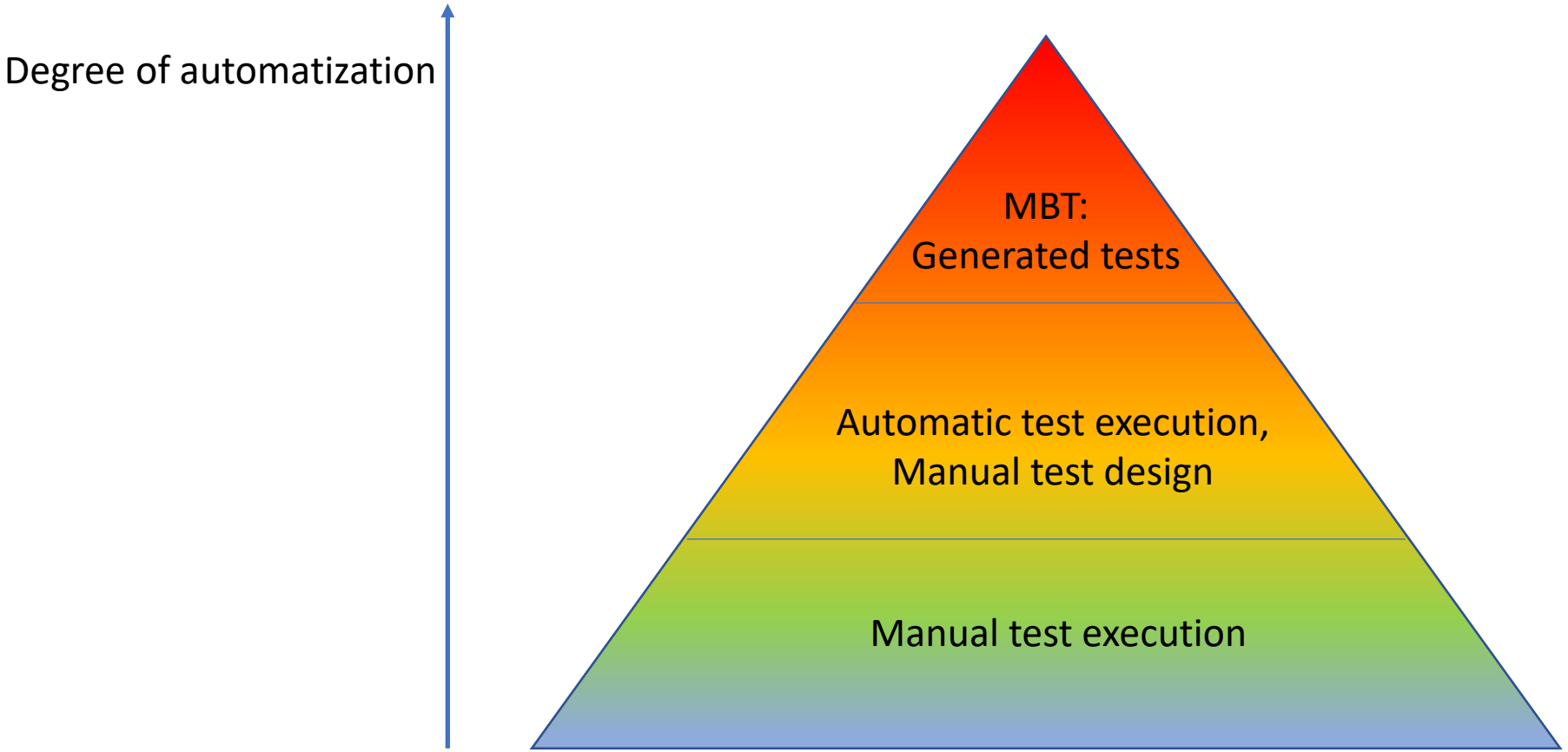
- Basic theory background of Model-based testing
- Finite State Machine models
- Editing specification models with GraphWalker Studio
- Generate basic test sequences with GraphWalker Studio and Model 》 Test 》 Relax

Degree of automatization



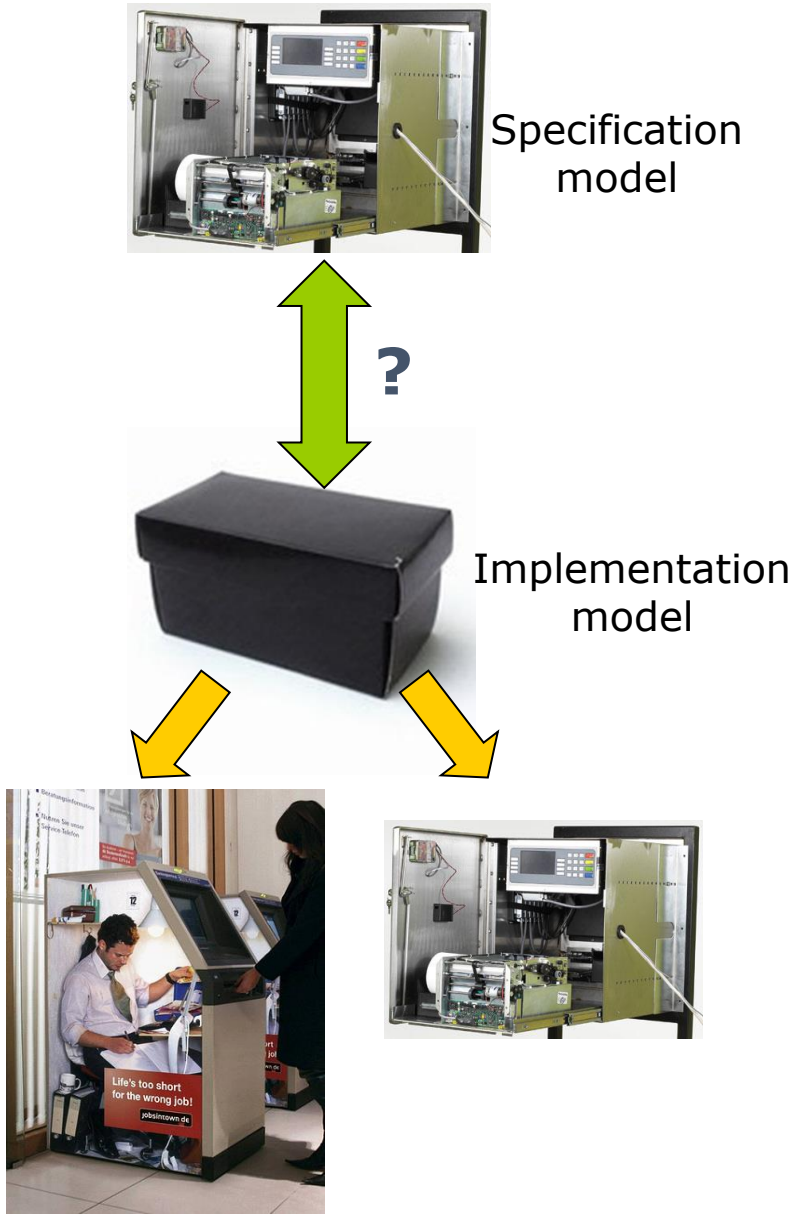
Degree of automatization





Model-based testing

- Conformance testing
  - Given
    - Specification model
      - *white box*
    - Implementation model
      - *black box*
  - Test if the implementation conforms to the specification?
    - Ie. determine if the two corresponding model descriptions are equivalent



FSM - a formal specification model

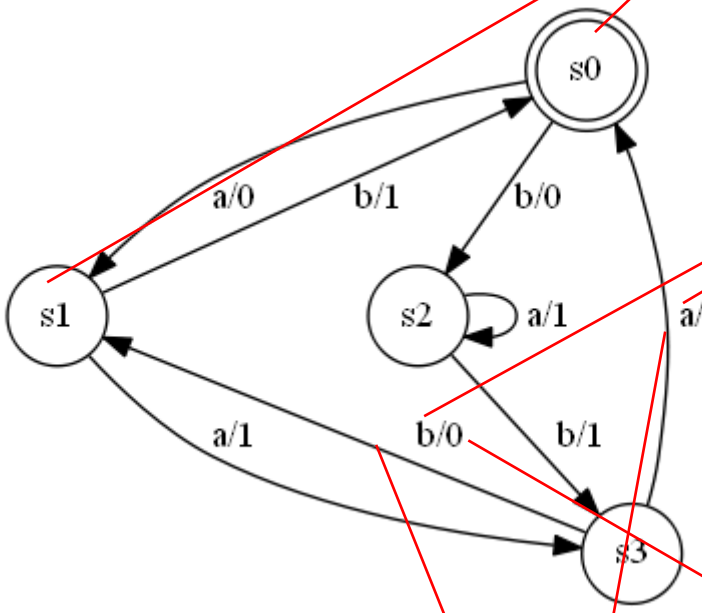
Finite State Machine

(Mealy-model)

- M=(I, O, S, T)
  - I: set of input symbols
  - O: set of output symbols
  - S: set of states
    - s_o: initial state
  - T: S x I → S x O: set of transitions
    - δ : S x I → S: next state function
    - λ : S x I → O: output function
    - Each t∈T: t=(s_x , s_y , i_x , o_x)

start state next state input output

state transition graph:



state transition table:

	a	b
s0	s1, 0	s2, 0
s1	s3, 1	s0, 1
s2	s2, 1	s3, 1
s3	s0, 0	s1, 0

states

inputs/events

transitions

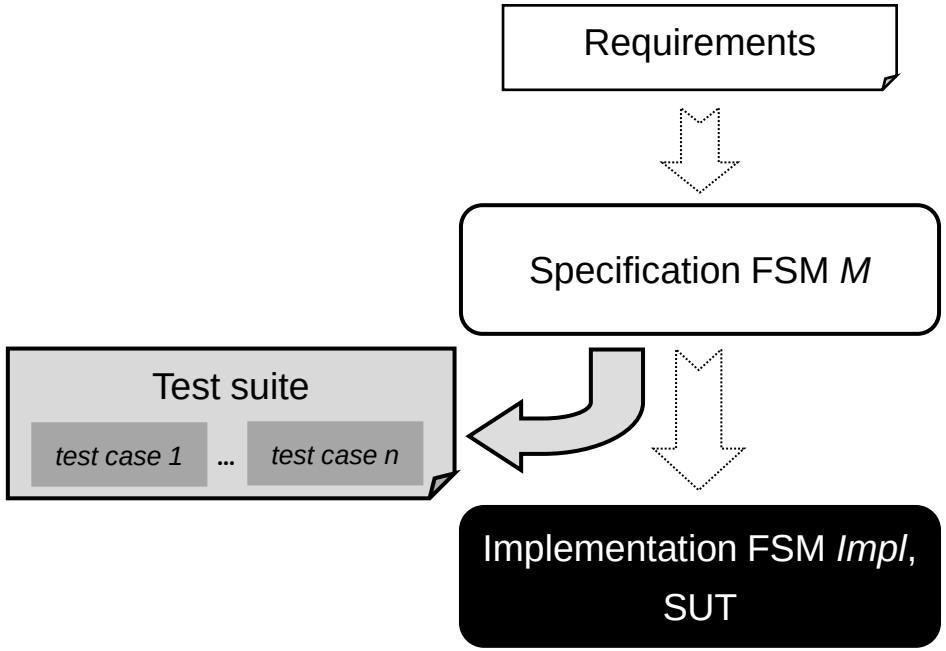
outputs/actions

FSM-based conformance testing

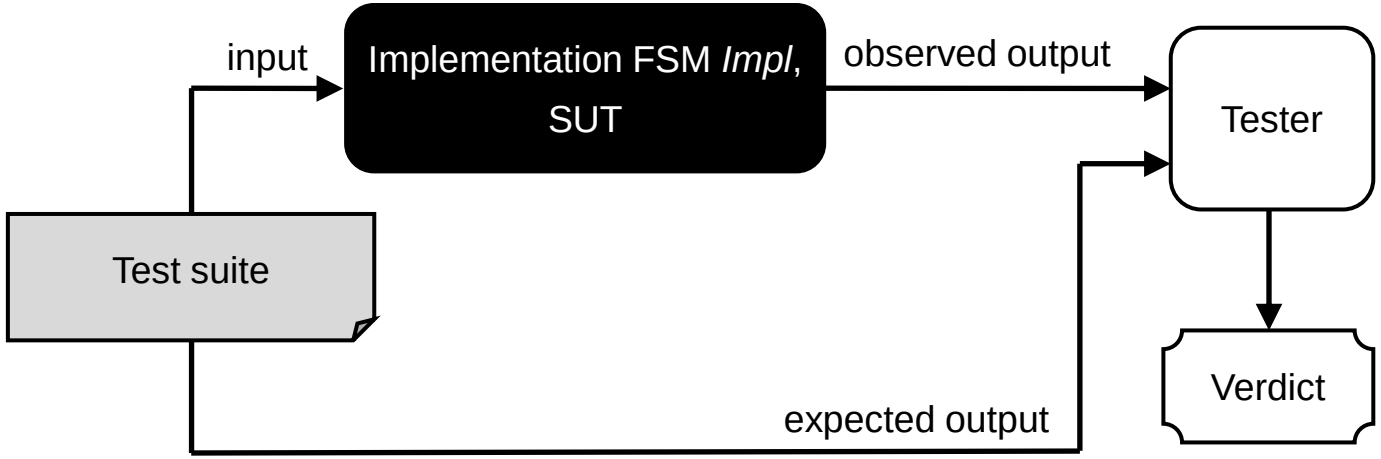
Specification:
FSM *M*
(white box)

?

Implementation:
FSM *Impl*
(black box)



FSM-based test generation



FSM-based testing

FSM exercise

Märklin Battery-Operated Signal

1. Explore the functionality of the given device & write a specification for it!

- The device has 2 LEDs (red and green), a 2-way switch and a press button.
- The 2-way switch turns the device off or on. When the device is turned on, the green LED lights up.
- The button can be used to change which LED lights; red or green.
- If no button is pressed for 7 secs the device changes the lights.

FSM exercise

Märklin Battery-Operated Signal

1. Explore the functionality of the given device & write a specification for it!

2. Create an FSM based on the specification!
   - Create a state transition table
   - Create a state transition graph

Waiting for 7 seconds can be handled as special `timeout` *input message*

state transition table:

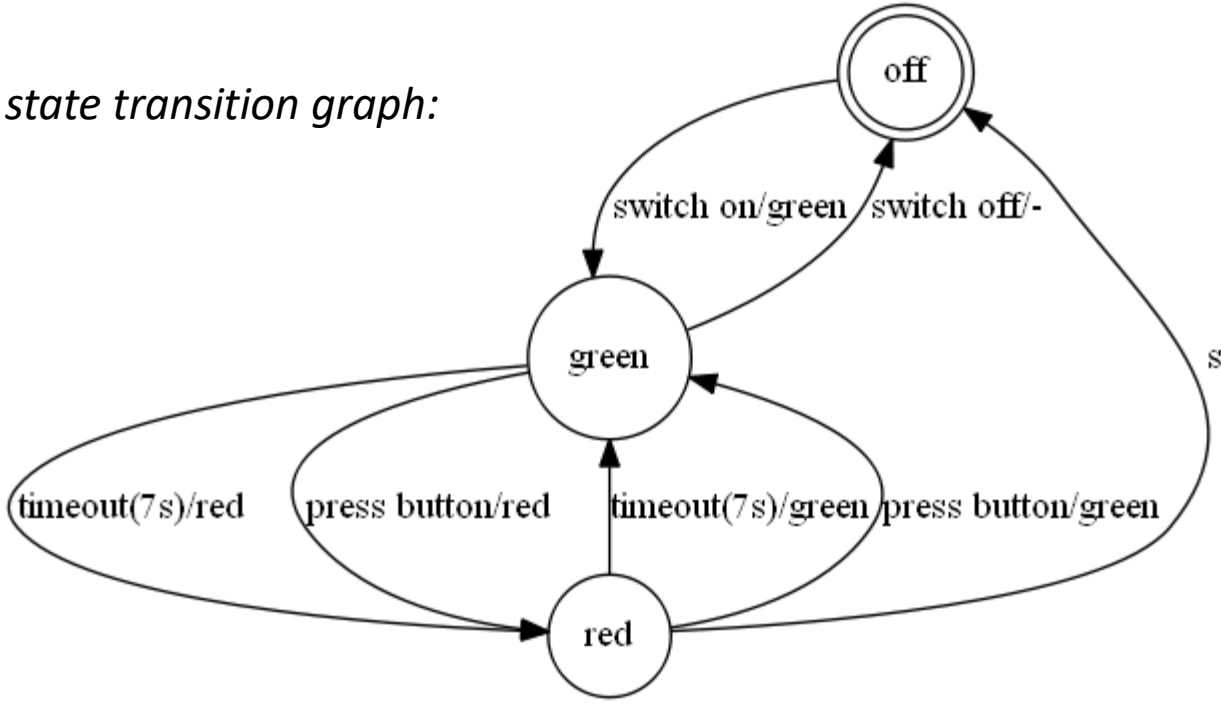
	inputs →			
states ↓				
		Next state / output		

FSM exercise: solution

state transition table:

	switch on	switch off	press button	wait for 7 secs
s_off	s_{green} /green	-	-	-
s_green	-	s_{off} /-	s_{red} /red	s_{red} /red
s_red	-	s_{off} /-	s_{green} /green	s_{green} /green

state transition graph:



FSM exercise

Märklin Battery-Operated Signal

1. Explore the functionality of the given device & write a specification for it!
2. Create an FSM based on the specification!
3. Design an input/output sequence to test all functionalities of the specification!

FSM exercise: solution

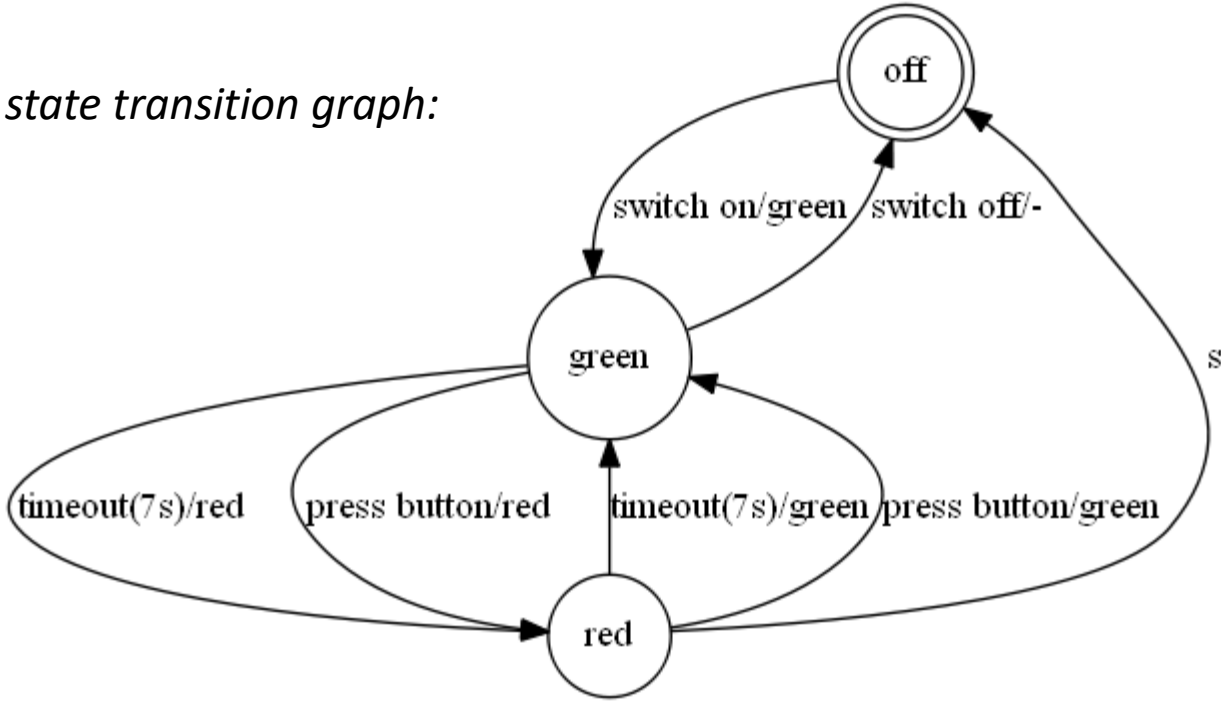
state transition table:

	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	-	-	-
green	-	s_{off} /-	s_{red} /red	s_{red} /red
red	-	s_{off} /-	s_{green} /green	s_{green} /green

example test sequence:

Action (input)	Desired output
switch on	green
wait for 7 secs	red
wait for 7 secs	green
press button	red
switch off	-
switch on	green
press button	red
press button	green
switch off	-

state transition graph:

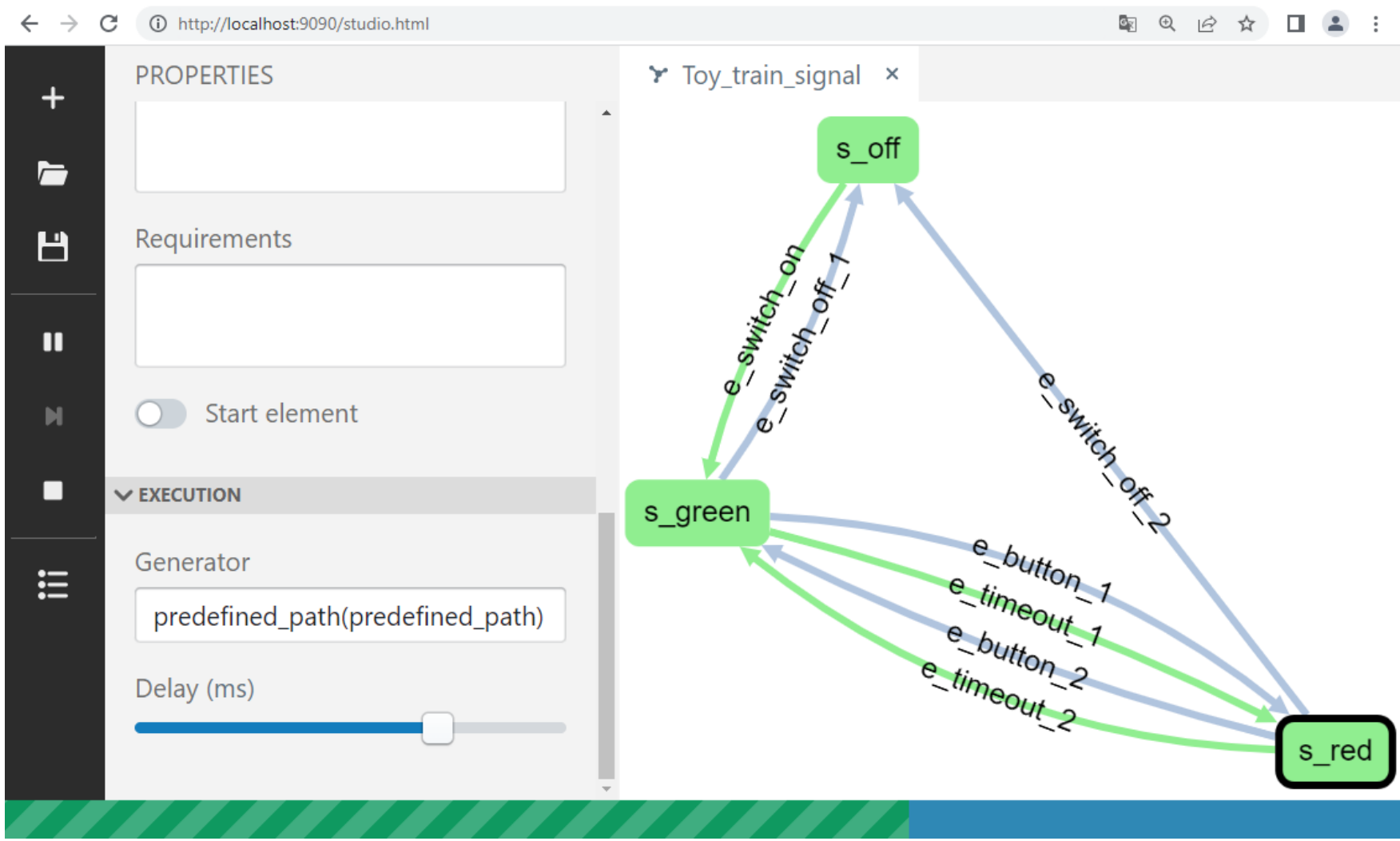


GraphWalker

- Open source, free MBT tool with GUI
- Easy-to-use
- Good documentation

Working process:

1. Creating a model
2. Write adaptation code that interacts with the SUT
3. Generate tests
4. Executing tests



GraphWalker Studio: Installation and start

I. Installation prerequisites:

Download and install Java JDK (or at least Java RE) and add to path

(using *JAVA_PATH* system environment variable in Windows)

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_192

II. Install model editor:

- Download GraphWalker Studio

After installation:

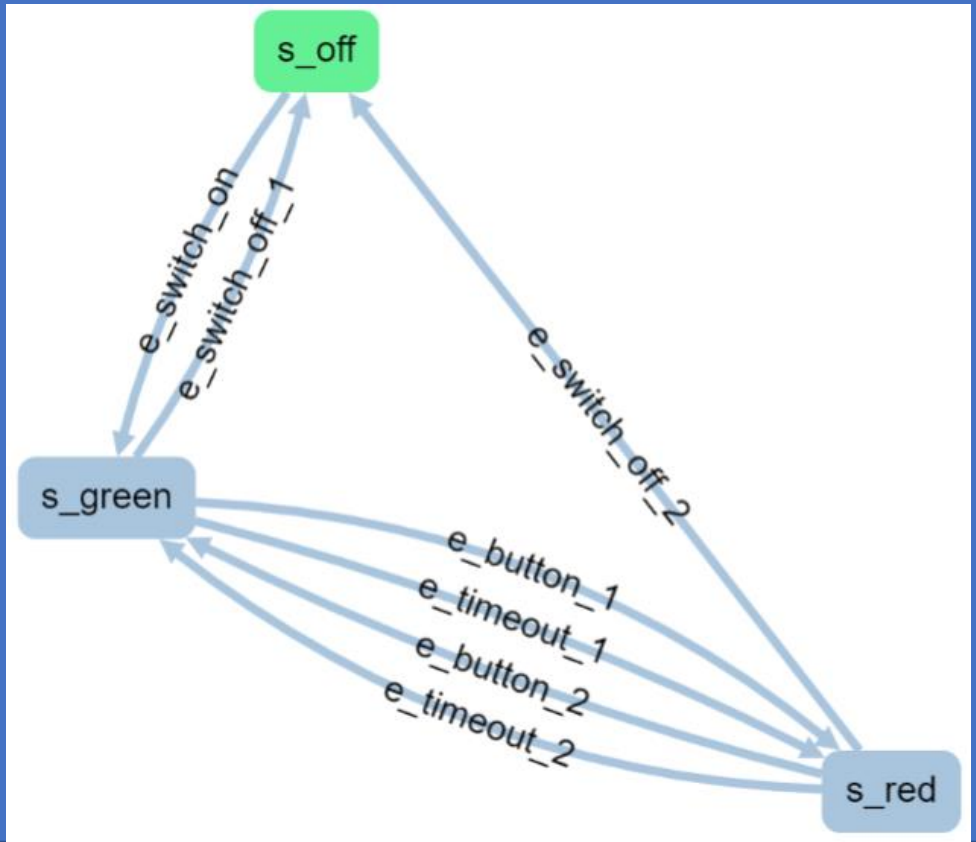
1. Click on GraphWalker Studio file *(graphwalker-studio-4.3.2.jar)*
2. Open a web browser and type http://localhost:9090/studio.html

GraphWalker Studio: Battery-operated signal

Draw the FSM of battery-operated signal in GW Studio

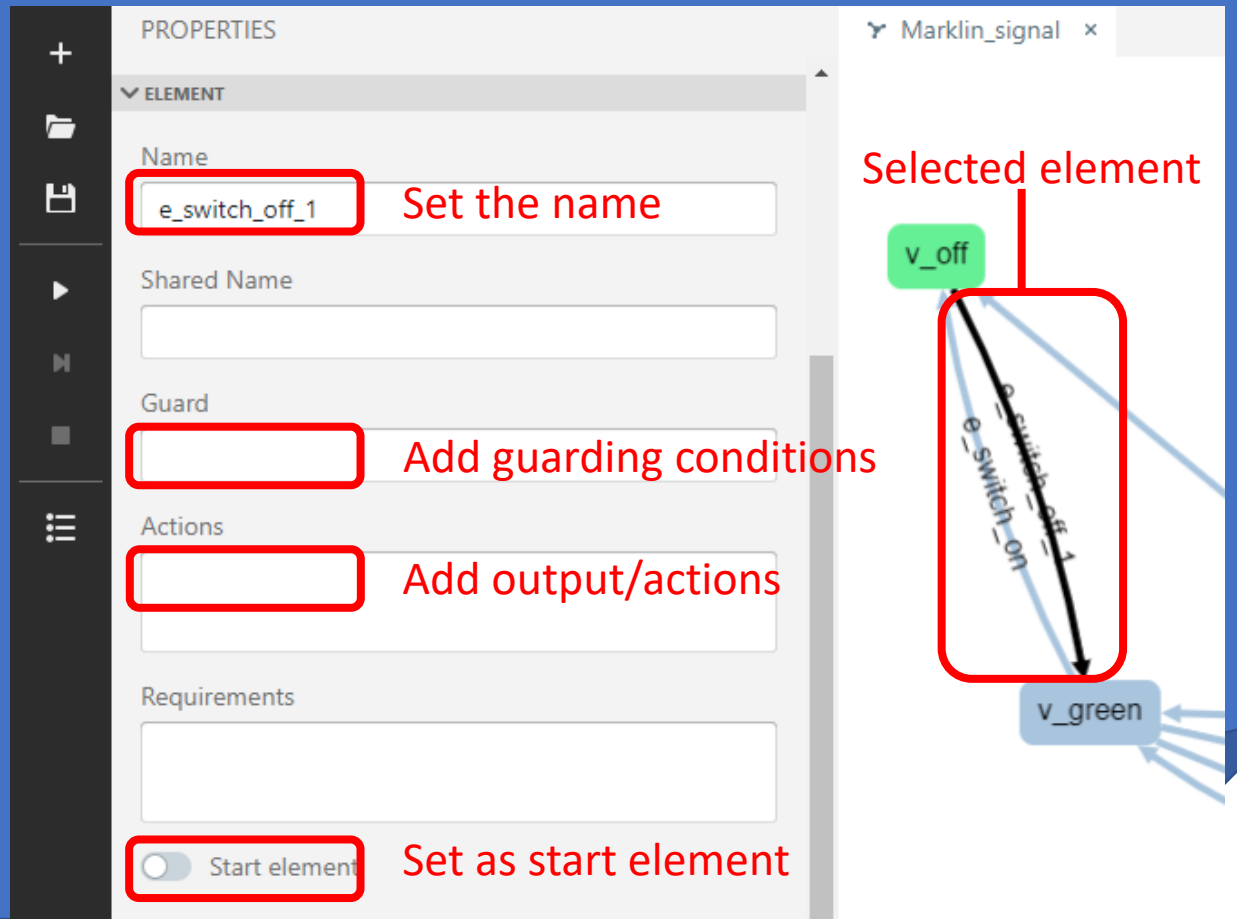
- Set up names for states (vertices) and transitions (edges)
- Set *s_off* as starting state
- At the current stage do not care outputs *(they can be saved as variables if you insist)*
- Save the model as *Toy_train_signal.json*

	switch on	switch off	press button	wait for 7 secs
s_off	s_green /green	-	-	-
s_green	-	s_off /-	s_red /red	s_red /red
s_red	-	s_off /-	s_green /green	s_green /green



GraphWalker Studio help 4/1: Create a model

- **Create a vertex**: On the editor area, while pressing the keyboard key *v*, click the *left mouse button*.
- **Create an edge**: On the editor area, while pressing the keyboard key *e*
  - *Click and hold the left mouse button on the first vertex.*
  - *Drag the mouse cursor to the second vertex and release the left mouse button over that vertex.*
- **Change the properties of a vertex/edge**:
  1. Select the given element with mouse click
  2. then click on ☰ at the left panel and
  3. Set the given property
     - Name
     - Guarding conditions
     - Output/actions
     - Start element or not?

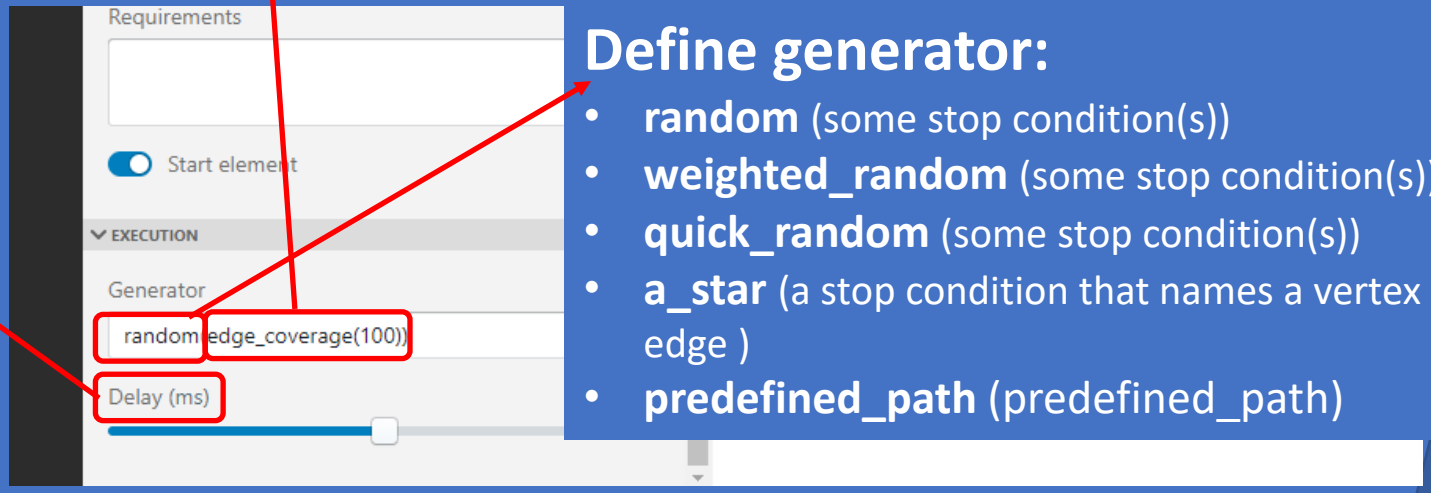


GraphWalker Studio help 4/2: Generators

Define stopping conditions for tests:

- **edge_coverage** (an integer representing percentage of desired edge coverage)
- **vertex_coverage** (an integer representing percentage of desired vertex coverage)
- **requirement_coverage** (an integer representing percentage of desired requirement coverage)
- **dependency_edge_coverage** (an integer representing dependency threshold)
- **reached_vertex** (the name of the vertex to reach)
- **reached_edge** (the name of the edge to reach)
- **time_duration** (an integer representing the number of seconds to run)
- **length** (an integer representing the total numbers of edge-vertex pairs)
- **never**

Define delays after each step



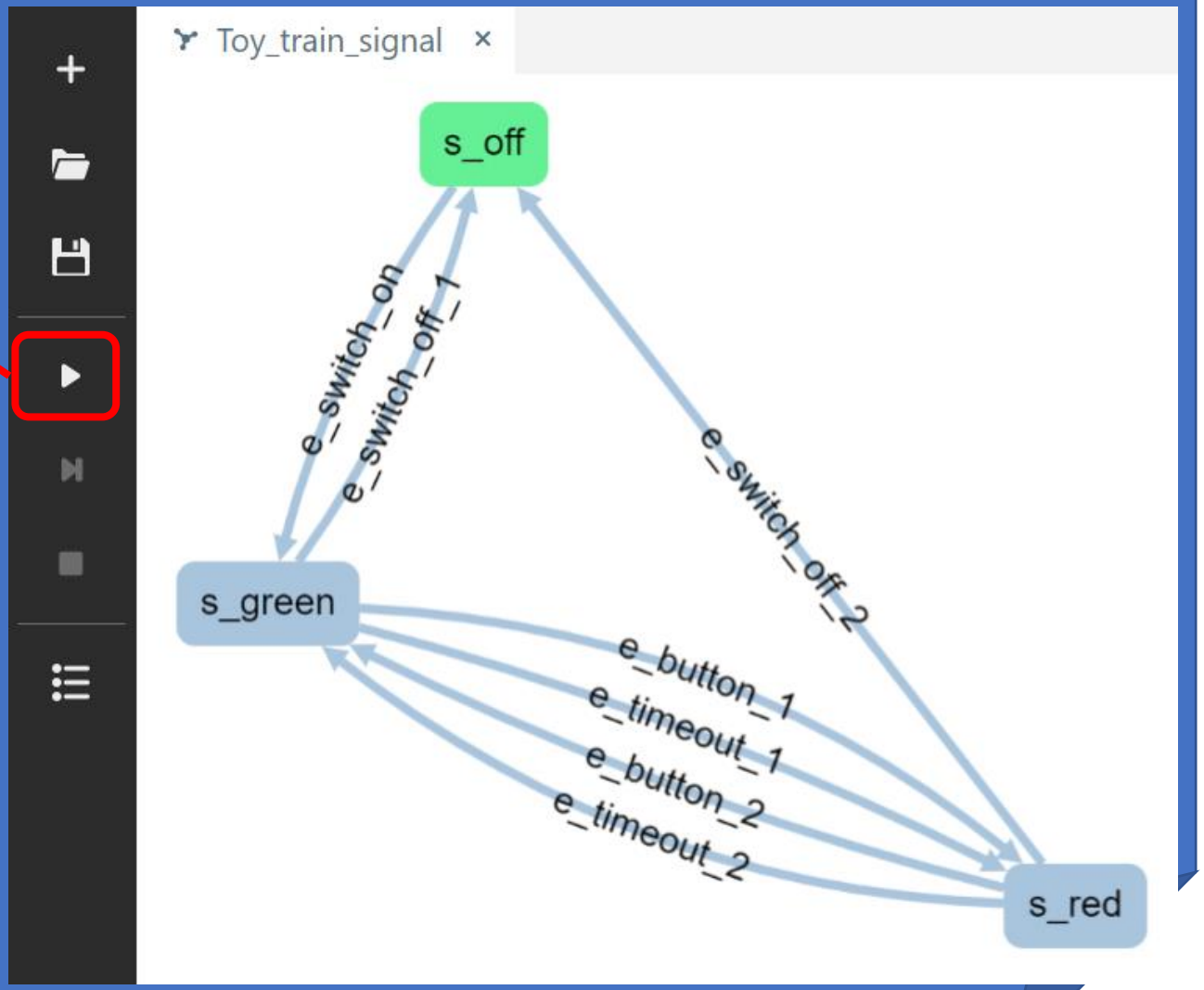
Define generator:

- **random** (some stop condition(s))
- **weighted_random** (some stop condition(s))
- **quick_random** (some stop condition(s))
- **a_star** (a stop condition that names a vertex or an edge)
- **predefined_path** (predefined_path)

More info in user guide

GraphWalker Studio help 4/3: Test execution

- Execute the tests:

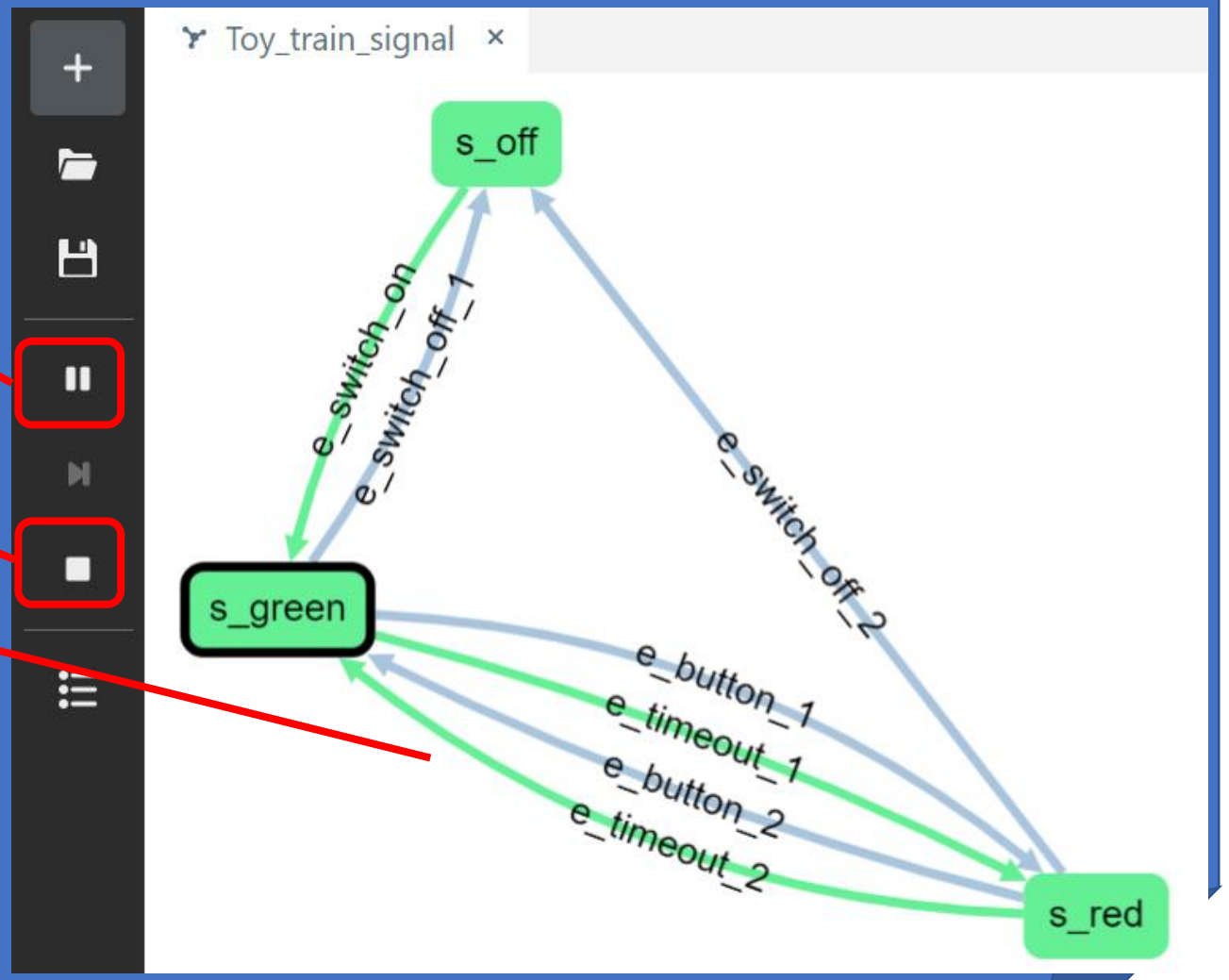


GraphWalker Studio help 4/4: Test execution

Pause test execution

Stop test execution

- Blue: unvisited transition / state
- Green: traversed transition / visited state
- Black: currently executes



GraphWalker Studio: Battery-operated signal

Run the tests in GW Studio

…using different generators:

- `random` / `quick_random` / `weighted_random` / `a_star`

- …using different stopping conditions:
  - `edge_coverage` / `vertex_coverage` (% of desired coverage)
  - `reached_vertex` / `reached_edge` (desired destination)
  - `time_duration` / `length`

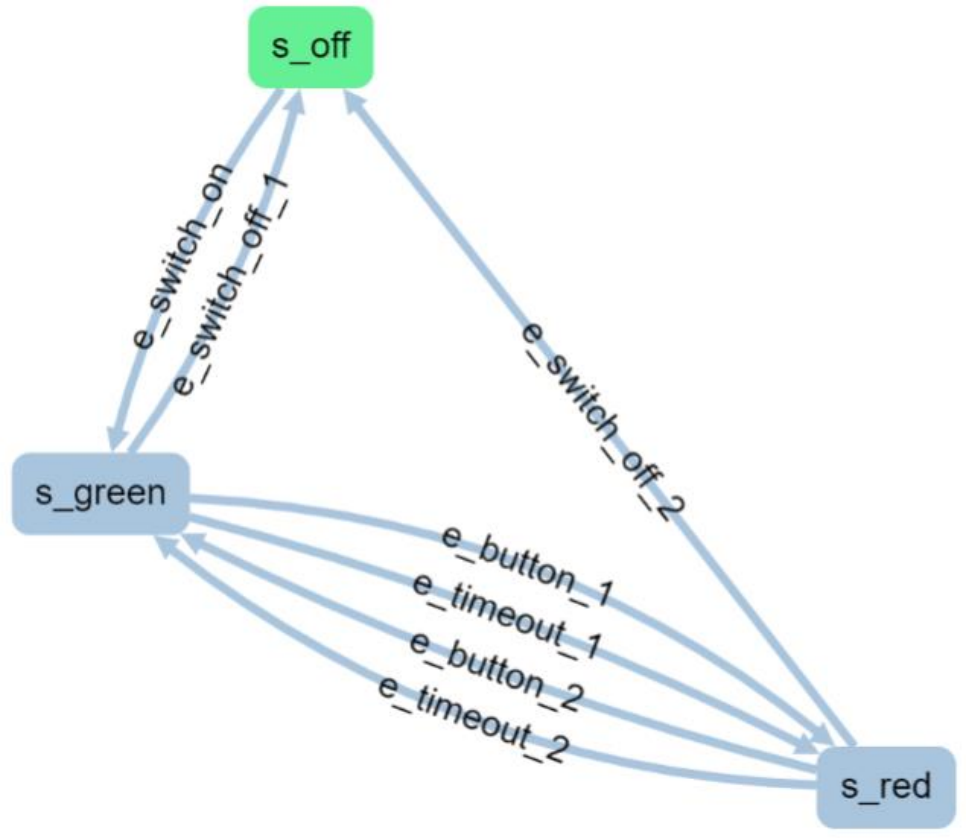
- Run the generator with given settings multiple times!
  - What can you observe? Why?

GraphWalker Studio: Battery-operated signal

- Some examples using different generators:

`weighted_random``(edge_coverage(100))`

- Same as `random`, but uses `weight` keyword (a float value between 0.0 and 1.0 can be assigned to edges of the graph) as a probability to select an edge when generate a path
  - If no `weight` keyword is used for edges, then the probabilities will be the same for these edges



Compared to `random` generator the length of test sequences are longer due the weighted distribution of edges (in my 5 conducted experiments they were **14, 24, 25, 40**, and **41**)

GraphWalker Studio: Battery-operated signal

- Some examples using different generators:

`quick_random``(edge_coverage(100))`

1. Choose an edge not yet visited by random and find the shortest path to that edge using Dijkstra's algorithm
2. Mark all edges in this park as visited then continue with step 1 till stopping condition fulfilled

- Gives reasonable short test sequences, but do not care the guarding conditions of EFSMs

- In my 5 conducted experiments `quick_random` traverses **9, 9, 11, 11** and **12** edges, respectively, to fulfill 100% edge cover condition, thus it is better than generator `random`. The optimal solution derived by TT-method requires only 9 edge traversals (the graph has 7 edges)
- Thus in multiple executions we can observe different test sequence lengths due random walking approach.

`a_star(reached_vertex(v_red))`

- Reaches a given vertex or edge (defined in stopping condition) with the least cost using the A* algorithm

```
"vertexCoverage": 66,
"totalNumberOfEdges": 7,
"totalNumberOfVisitedVertices": 2,
"edgeCoverage": 28,
"totalNumberOfVertices": 3,
"totalNumberOfUnvisitedEdges": 5
```

Screenshot from GraphWalker CLI version

GraphWalker Studio: Battery-operated signal

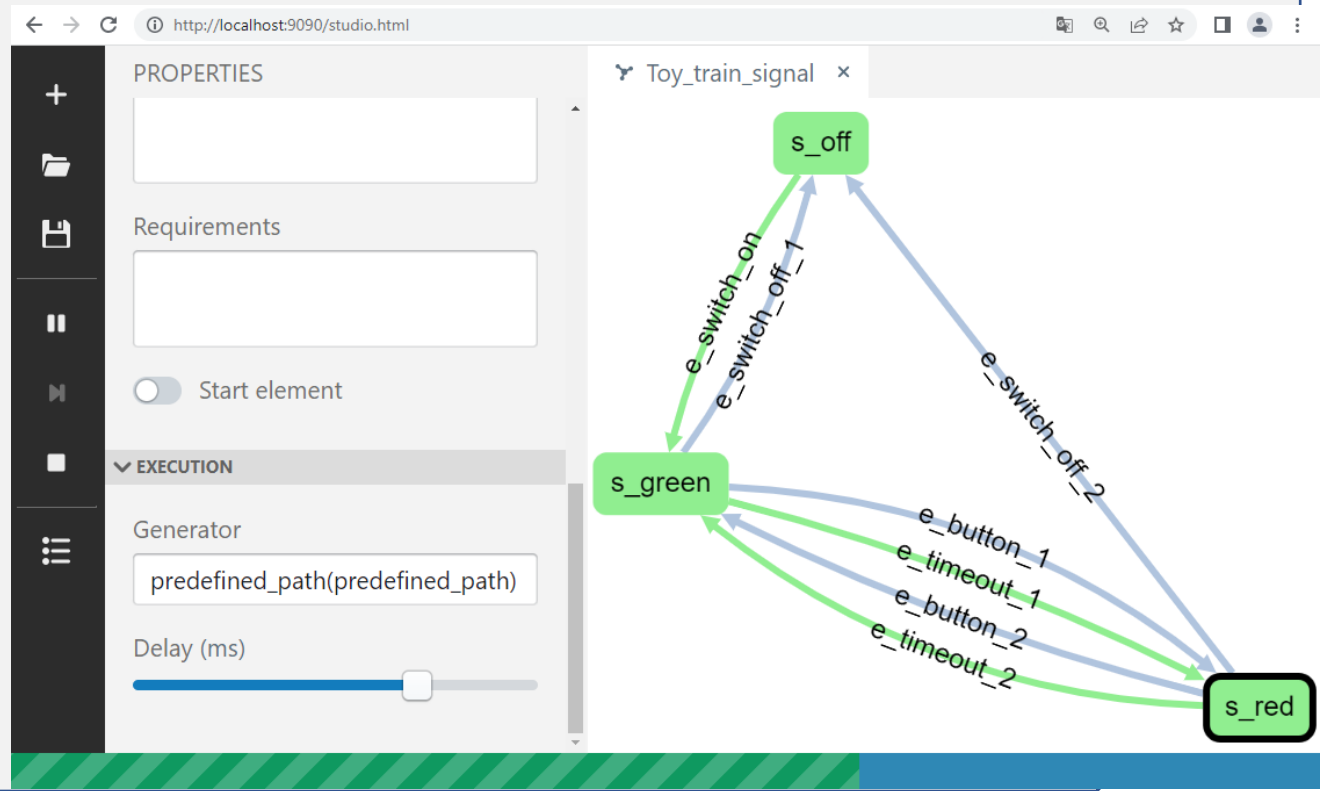
- Some examples using different generators:

`predefined_path(predefined_path)`

Enables the user to define an edge sequence that GraphWalker will use when executing. The predefined path can be specified by the user manually in the model file, or the test suites generated by Model 》 Test 》 Relax can be added to the model using the `--predefined_path` flag of Model 》 Test 》 Relax

(This functionality has been added to GraphWalker from Model 》 Test 》 Relax side).

```
{
   "models":[
      {
          "generator": "predefined_path(predefined_path)",
         "name":"Toy_train_signal",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-670cd27f6cbe",
         "vertices":[
                                 ...
         ],
         "edges":[
                                 ...
         ],
         "predefinedPathEdgeIds":[
           "e1,,, "e3", "e5", "e3", "e4",
           "e0", "e2", "e1", "e6"
         ]
      }
   ]
}
```



GraphWalker Studio: Summary of Random Generators

- Random tests
  - Can be useful for exploratory testing
  - Not repeatable
    - No suitable for regression tests → suitable only for educational purposes/small projects/augmenting of other testing infrastructure
  - Tests are not optimal
    - Impractical for the functional testing of a large-scale software

Some systematic test generation algorithms

n: number of states
p: number of input symbols
m: number of transitions

Test generation algorithm	Complexity of test generation	Complexity of test suite	Structure of test suite	Coverage	Notes
All-State (AS)	$O(n^2)$	$O(m)$	1 test sequence	100% state coverage	Heuristic algorithm
Transition Tour (TT)	$O(n^3+m)$	$O(m)$	1 test sequence	100% state- and transition coverage. It guarantees to discover all output faults, but does not guarantee to find transfer faults	The resulting test sequence is the shortest possible tour that provides 100% state- and transition coverage
All-Transition-State (ATS)	$O(n^3+m)$ (ATS0 version), $O(\eta(n^3+m))$ (iterative version),	$O(m)$ (ATS0 version), $O(\eta \cdot m)$ (iterative version),	1 test sequence (with subparts)	It guarantees to discover all output faults and it finds virtually all transfer faults	Heuristic algorithm
Harmonized State Identifiers (HSI)	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Structured test suite with multiple test sequences	It guarantees to discover output and transfer faults and given number of extra states in implementation	An improved version of this method exist, called H-method
H-method	$O(p \cdot n^3)$	$O(p \cdot n^3)$	Structured test suite with multiple test sequences	It guarantees to discover output and transfer faults and given number of extra states in implementation	Improvement of the HSI-method
N-Switch Coverage	$O((N+1) \cdot m^{(k+1) \cdot (N+1)})$, where $k=[0..N]$ iteration limit	$O((N+1) \cdot m^{N+1})$	1 test sequence	It covers all topologically possible, consecutive $N+1$ transitions	The test generation time and the size of the resulting test sequence dramatically increases if N or the size of the input model increases!

Model 》 Test 》 Relax

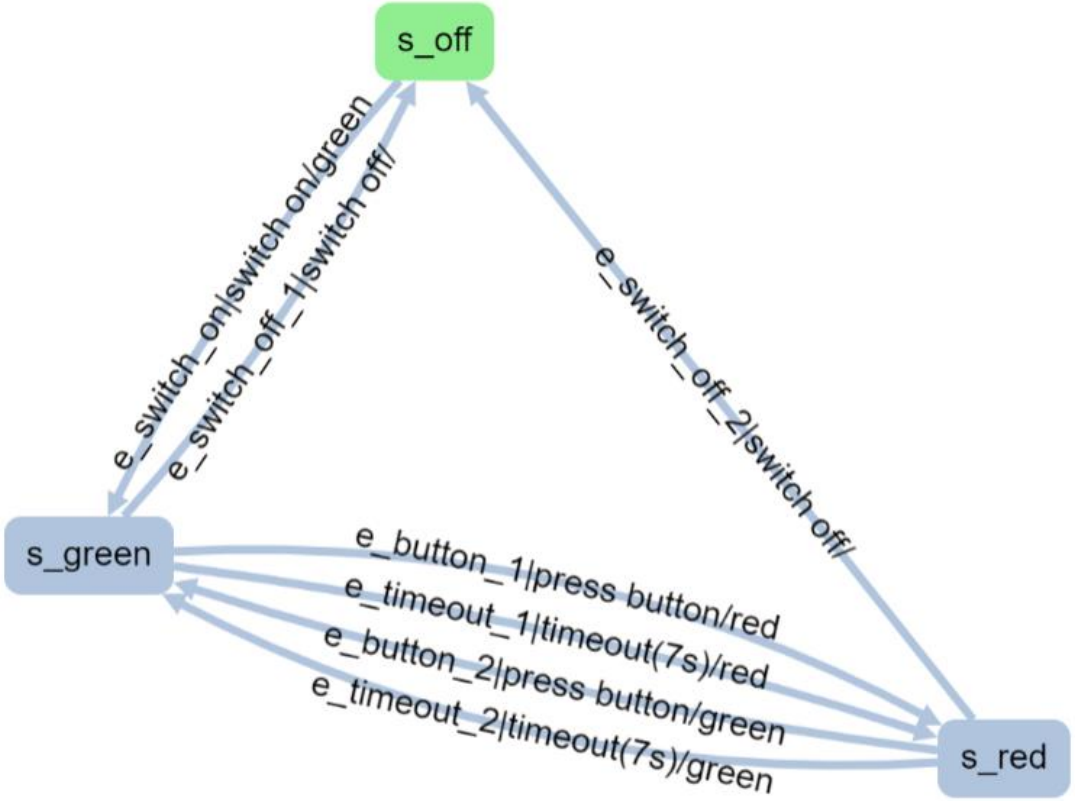
- Open source, free MBT tool
- Developed in ELTE Faculty of Informatics
- Easy-to-use

- Models can be imported from GraphWalker Studio
- Has systematic test generation algorithms, model conversions
- Has good user guide and sample test projects

Download executable from here

Model 》 Test 》 Relax: Import model from GraphWalker Studio

Models created with GraphWalker Studio does not handle input and output symbols, while Model 》 Test 》 Relax does.
There are 2 options to handle this:
1. Add i/o symbols to the model encoded in the edge names using the following format: `edge_name|input/output`



*(ASCII code for | and / characters are **124** and **047**, respectively; hold down ALT while type the character code using the numeric keypad).*

Then you can apply [Add input/output symbols to the model functionality](#) of Model 》 Test 》 Relax:

```
./MTR -o conversion -m add_iosymbols -f <path>/<name of your model>
```

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[C-IO   ] [warning] The following transition does not have output symbol encoded in edge name: e_switch_off_2|switch off/
[C-IO   ] [warning] The following transition does not have output symbol encoded in edge name: e_switch_off_1|switch off/
[C-IO   ] [info] Specified I/O symbols has been added to model: converted/Toy_train_signal_add_io_example-iosymbols.json
```

These warnings are expected as we did not define outputs for switch off transitions

M▸T▸R *Note that the converted model is included in the [Model 》 Test 》 Relax](#) repository: See `sample_models/introduction/Toy_train_signal.json`*

Model 》 Test 》 Relax: Import model from GraphWalker Studio

OR:

2. Add input/output symbols manually into in the *`Toy_train_signal.json`* JSON model (see "edges" in model file) after the "input" and "output" keywords:

```
{
   "models":[
      {
         "name":"Toy_train_signal",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-
670cd27f6cbe",
         "vertices":[
            {
               "id":"n0",
               "name":"s_green",
               "properties":{
                  "x":20,
                  "y":263.5
               }
            },
            {
               "id":"n1",
               "name":"s_red",
               "properties":{
                  "x":364,
                  "y":392
               }
            },
            {
               "id":"n2",
               "name":"s_off",
               "properties":{
                  "x":112,
                  "y":68
               }
            }
         ],
         "edges":[
            {
               "id":"e0",
               "name":"e_button_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "press button",
               "output" : "red",
               "weight": "0.3"
            },
            {
               "id":"e1",
               "name":"e_switch_on",
               "sourceVertexId":"n2",
               "targetVertexId":"n0",
               "input" : "switch on",
               "output" : "green",
               "weight": "1"
            },
            {
               "id":"e2",
               "name":"e_switch_off_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight": "0.1"
            },
            {
               "id":"e3",
               "name":"e_timeout_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "timeout(7s)",
               "output" : "red",
               "weight": "0.6"
            },
            {
               "id":"e4",
               "name":"e_button_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "press button",
               "output" : "green",
               "weight": "0.3"
            },
            {
               "id":"e5",
               "name":"e_timeout_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "timeout(7s)",
               "output" : "green",
               "weight": "0.6"
            },
            {
               "id":"e6",
               "name":"e_switch_off_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight": "0.1"
            }
         ]
      }
   ]
}
```

Note that this model is included in the [Model 》 Test 》 Relax](#) repository: See `sample_models/introduction/Toy_train_signal.json`

Model 》 Test 》 Relax: Systematic test generation

- Transition Tour (TT):
  - Shortest tour that provides 100 state and transition coverage
  - Based on Directed Chinese Postman Problem

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --transition_list
```

Note that the --transition_list option add an optional transition name/id list to the test cases, but not required for test generation

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: No
[U-CTRL ] [info] Running Transition Tour test generation
[TG-TT  ] [info] Not Eulerian, augment
[TG-TT  ] [info] Building bipartite graph
[TG-TT  ] [info] Bipartite graph successfully built
[TG-TT  ] [info] Creating matching
[TG-TT  ] [info] Matching done
[TG-TT  ] [info] Duplicating transitions according to matching
[TG-TT  ] [info] Augmenting to Eulerian graph successful
[TG-TT  ] [info] Ordering edges
[TG-TT  ] [info] Edges ordered
[TG-TT  ] [info] Generating test sequence
[TG-TT  ] [info] Starting traversal
[TG-TT  ] [info] Tour length: 9
[TG-TT  ] [info] Finished computation at 2024-03-19 11:21:19.1432804
        elapsed time: (real time) 0.0034409 s
        elapsed time: (user time) 0.004000 s
[TG-TGR ] [info] Test generation summary written: test_summary/tt_result.csv
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-TT-test_suite.json
```

Model 》 Test 》 Relax: Systematic test generation

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --graphviz --transition_list
```

Create output to visualize models with Graphviz

Add transition lists into test suite

Console output:

```
[U-CTRL ] [info] Version: 3.3.5 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: No
[U-CTRL ] [info] Created graphviz dot file of original model: ./graphviz-TT
[U-CTRL ] [info] Running Transition Tour test generation
[TG-TT  ] [info] Not Eulerian, augment
[TG-TT  ] [info] Building bipartite graph
[TG-TT  ] [info] Bipartite graph successfully built
[TG-TT  ] [info] Creating matching
[TG-TT  ] [info] Matching done
[TG-TT  ] [info] Duplicating transitions according to matching
[TG-TT  ] [info] Augmenting to Eulerian graph successful
[TG-TT  ] [info] Ordering edges
[TG-TT  ] [info] Edges ordered
[TG-TT  ] [info] Generating test sequence
[TG-TT  ] [info] Starting traversal
[TG-TT  ] [info] Tour length: 9
[TG-TT  ] [info] Augmented model graphviz file created: graphviz/graphviz-TT/Toy_train_signal_augmented.dot
[TG-TT  ] [info] Finished computation at 2024-02-19 15:06:06.5910571
        elapsed time: (real time) 0.0039878 s
        elapsed time: (user time) 0.004000 s
[TG-TGR ] [info] Test generation summary written: test_summary/tt_result.csv
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-TT-test_suite.json
```

Short summary about input model

Short summary about programs settings

TG: Test generator component

Main steps of test generation

Short summary about the resulting test sequence

Test summary csv file is saved here

Resulting test suite is saved here

Model 》 Test 》 Relax: Systematic test generation

- Open a generated test sequence in *test_suites* folder (`Toy_train_signal-TT_suite.json` → *Transition Tour)*:


```
{
    "test_suite": {
        "name": "Toy_train_signal",
        "id": "08414683-68cb-4f68-be4c-670cd27f6cbe",
        "method": "TT",
        "input_list": [
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "timeout(7s)",
            "press button",
            "press button",
            "switch off",
            "switch on",
            "switch off"
        ],
        "output_list": [
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            ""
        ],
        "transition_id_list": [
            "e1",
            "e3",
            "e5",
            "e3",
            "e4",
            "e0",
            "e2",
            "e1",
            "e6"
        ],
        "transition_name_list": [
            "e_switch_on",
            "e_timeout_1",
            "e_timeout_2",
            "e_timeout_1",
            "e_button_2",
            "e_button_1",
            "e_switch_off_2",
            "e_switch_on",
            "e_switch_off_1"
        ]
    }
}
```


Name of the model

Used test generation algorithm

List of applied input symbols

List of desired output symbols derived from specification model

Transition lists are optional (see `--transition_list` switch) It can be used for learning/self checking or if one would like to import the results to GraphWalker

Model 》 Test 》 Relax

- Apply the generated test suite into the System Under Test


```
{
    "test_suite": {
        "name": "Toy_train_signal",
        "id": "08414683-68cb-4f68-be4c-670cd27f6cbe",
        "method": "TT",
        "input_list": [
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "timeout(7s)",
            "press button",
            "press button",
            "switch off",
            "switch on",
            "switch off"
        ],
        "output_list": [
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            ""
        ],
        "transition_id_list": [
            "e1",
            "e3",
            "e5",
            "e3",
            "e4",
            "e0",
            "e2",
            "e1",
            "e6"
        ],
        "transition_name_list": [
            "e_switch_on",
            "e_timeout_1",
            "e_timeout_2",
            "e_timeout_1",
            "e_button_2",
            "e_button_1",
            "e_switch_off_2",
            "e_switch_on",
            "e_switch_off_1"
        ]
    }
}
```


Output State (cannot be observed)

green

Play

Input	Output
switch on	green
timeout(7s)	red
timeout(7s)	green
timeout(7s)	red
press button	green
press button	red
switch off	
switch on	green
switch off	

Import

Insert the input_list and output_list of generated test suite here:


```
{
        "input_list": [
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "timeout(7s)",
            "press button",
            "press button",
            "switch off",
            "switch on",
            "switch off"
        ],
        "output_list": [
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            ""
        ]
}
```

Model 》 Test 》 Relax: Systematic test generation

- All-State (AS):
  - Traverses all states using Nearest Neighbour heuristic

```
./MTR -m AS -f sample_models/introduction/Toy_train_signal.json --transition_list
```

Note that the --transition_list option add an optional transition name/id list to the test cases, but not required for test generation

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: No
[U-CTRL ] [info] Running All-State test generation
[TG-TGR ] [info] Test generation summary written: test_summary/as_result.csv
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-AS-test_suite.json
```

Model 》 Test 》 Relax: Systematic test generation

- All-Transition-State (ATS):
  - Provides a tour that traverses all transitions, then all states and then tries to find alternative routes

```
./MTR -m ATS -f sample_models/introduction/Toy_train_signal.json --transition_list
```

Note that the --transition_list option add an optional transition name/id list to the test cases, but not required for test generation

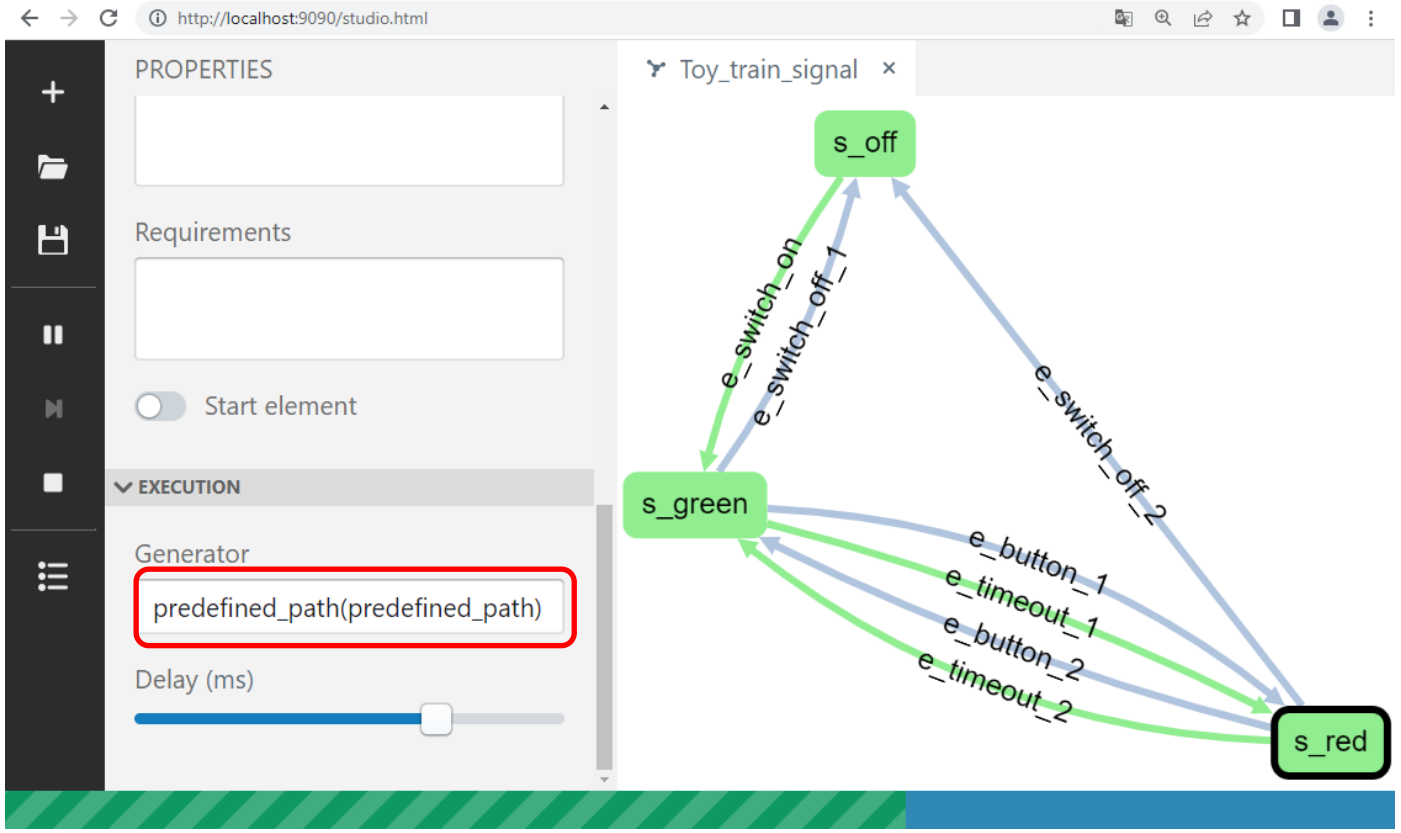
```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: No
[U-CTRL ] [info] Running All-Transition-State test generation, depth: 0
[TG-AT  ] [info] Building bipartite graph
[TG-AT  ] [info] Bipartite graph successfully built
[TG-AT  ] [info] Creating matching
[TG-AT  ] [info] Matching done
[TG-AT  ] [info] Duplicating the paths represented by the edges in the matching
[TG-AT  ] [info] Augmenting to Eulerian graph successful
[TG-AT  ] [info] Ordering edges
[TG-AT  ] [info] Transitions ordered
[TG-AT  ] [info] Traversal: Initial state: s_off, transition count: 9
[TG-AT  ] [info] AT traversal stopped on state: s_off
[TG-ATS0] [info] Spanning tree with paths to the leaf nodes built, num of arcs: 4, total: 7
[TG-ATS0] [info] Running all transition algorithm on extended spanning tree
[TG-AT  ] [info] Building bipartite graph
[TG-AT  ] [info] Bipartite graph successfully built
[TG-AT  ] [info] Creating matching
[TG-AT  ] [info] Matching done
[TG-AT  ] [info] Duplicating the paths represented by the edges in the matching
[TG-AT  ] [info] Augmenting to Eulerian graph successful
[TG-AT  ] [info] Ordering edges
[TG-AT  ] [info] Transitions ordered
[TG-AT  ] [info] Traversal: Initial state: s_red, transition count: 5
[TG-AT  ] [info] AT traversal stopped on state: s_red
[TG-ATS0] [warning] Complement CT graph not strongly connected, alternate sequences won't be completely arc disjoint.
[TG-ATS0] [info] SC graph transition count: 0
[TG-ATS0] [info] Successfully restored strongly connected property transitions used: 2
[TG-AT  ] [info] Building bipartite graph
[TG-AT  ] [info] Bipartite graph successfully built
[TG-AT  ] [info] Creating matching
[TG-AT  ] [info] Matching done
[TG-AT  ] [info] Duplicating the paths represented by the edges in the matching
[TG-AT  ] [info] Augmenting to Eulerian graph successful
[TG-AT  ] [info] Ordering edges
[TG-AT  ] [info] Transitions ordered
[TG-AT  ] [info] Traversal: Initial state: s_green, transition count: 7
[TG-AT  ] [info] AT traversal stopped on state: s_green
[TG-ATS0] [info] Result:
        OG AT: 9, OG TSP: 2
        ST AT: 5, ST TSP: 2
        CST AT: 7, CST TSP: 2

[TG-ATS0] [info] Finished computation at 2024-03-19 11:23:36.3689180
        elapsed time: (real time) 0.0348343 s
```

Model 》 Test 》 Relax: Import generated tests into GraphWalker

1. Generate test suite with the Transition Tour (TT) algorithm on the toy train signal example model using the `--predefined_path` flag:

```
./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --predefined_path
```

2. Execute the test suite in GraphWalker Studio.



Model 》 Test 》 Relax: Import test suites into GraphWalker


```
{
   "models":[
      {
        "generator": "predefined_path(predefined_path)",
         "name":"Toy_train_signal",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-
670cd27f6cbe",
         "vertices":[
            {
               "id":"n0",
               "name":"s_green",
               "properties":{
                  "x":20,
                  "y":263.5
               }
            },
            {
               "id
               "na
               "pr
               }
            },
            {
               "id":"n2",
               "name":"s_off",
               "properties":{
                  "x":112,
                  "y":68
               }
            }
         ],
         "edges":[
            {
               "id":"e0",
               "name":"e_button_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "press button",
               "output" : "red"
            },
            {
               "id":"e1",
               "name":"e_switch_on",
               "sourceVertexId":"n2",
               "targetVertexId":"n0",
               "input" : "switch on",
               "output" : "green"
            },
               tch_off_2",
               Id":"n1",
               Id":"n2",
               itch off",
            },
            {
               "id":"e3",
               "name":"e_timeout_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "timeout(7s)",
               "output" : "red"
            },
            {
               "id":"e4",
               "name":"e_button_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "press button",
               "output" : "green"
            },
            {
               "id":"e5",
               "name":"e_timeout_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "timeout(7s)",
               "output" : "green"
            },
            {
               "id":"e6",
               "name":"e_switch_off_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : ""
            }
         ],
         "predefinedPathEdgeIds":[
           "e1",
           "e3",
           "e5",
           "e3",
           "e4",
           "e0",
           "e2",
           "e1",
           "e6"
         ]
      }
   ]
}
```


This is the `transition_id_list` part of the test sequence generated by Model 》 Test 》 Relax using the `./MTR -o t -m tt -f sample_models/introduction/Toy_train_signal.json --transition_list` command

Part 2

- Extended Finite State Machine models
- Reset messages
- Test generation algorithms with multiple test sequences

Extensions of the FSM formalism: EFSMs

- EFSMs (Extended Finite State Machines)
  - The extension of FSMs with variables and operations based on variable values

- M=(I, O, S, V, T)
  - I: set of input symbols
  - O: set of output symbols
  - S: set of states
  - V: set of variables (each of it with given range)
  - T: set of transitions
    - Each $t \in T$: $t=(s_x, s_y, i_x, o_x, P_x(V), A_x(V))$

start state — next state — input — output — predicate on current variable values — action on current variable values

Extensions of the FSM formalism: EFSMs

The predicate on variable values of the EFSM can be denoted…

1. as an input of the transition originating from a distinct predicate state

OR

2. as a part of an input (called GUARD condition, written between "[ ]" brackets) of the transition originating from the given state

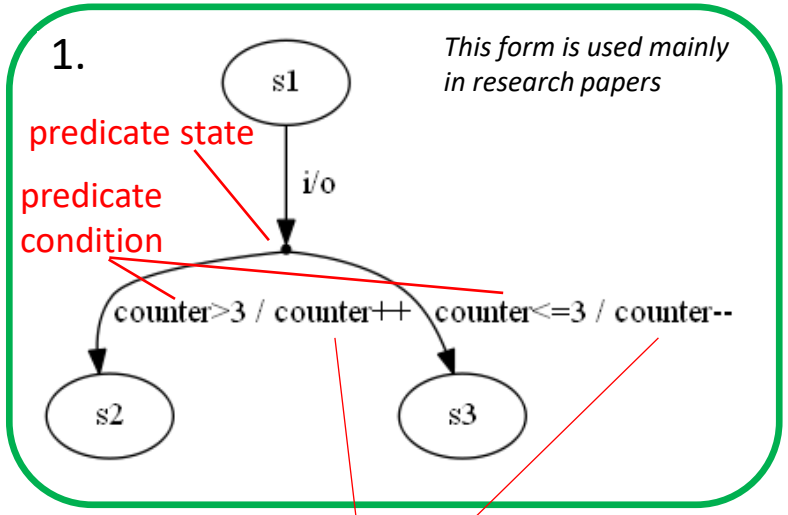
Similarly the action on current variable values can be…

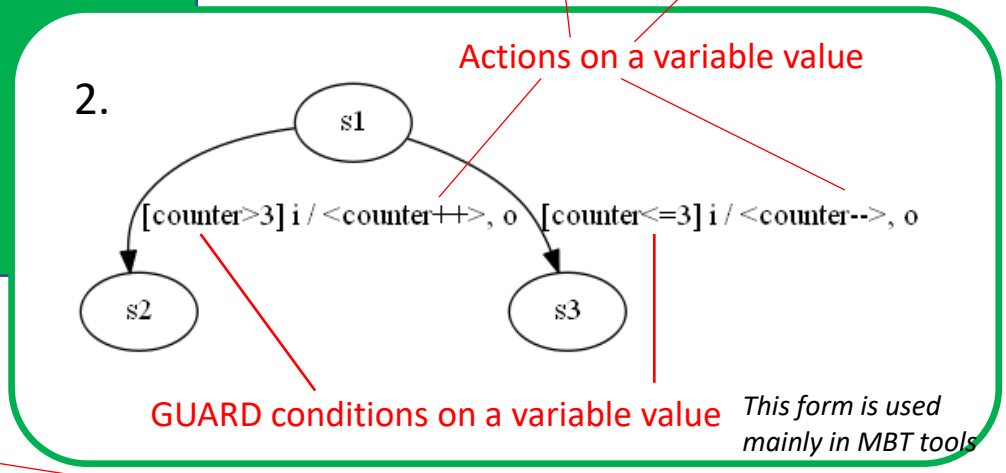
1. denoted as an output of the transition originating from a distinct predicate state

OR

2. as a part of an output of the transition originating from the given state

The 2 description modes can be transformed into each other



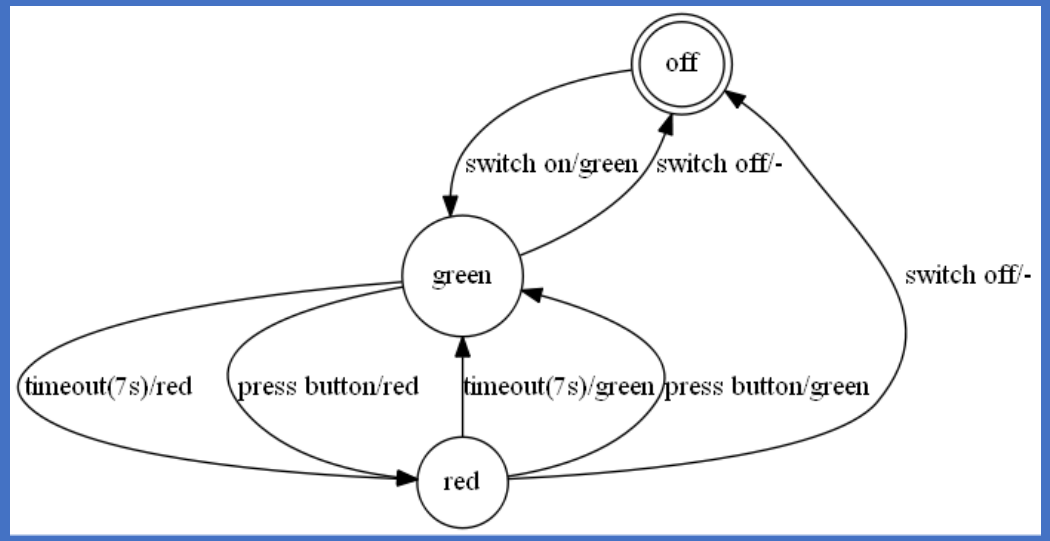


- T: set of transitions
  - Each $t \in T$: $t=(s_x, s_y, i_x, o_x, P_x(V), A_x(V))$

start state — next state — input — output — predicate on current variable values — action on current variable values

EFSMs - exercise

Battery-Operated Signal – extended version



	switch on	switch off	press button	wait for 7 secs
off	s_{green} /green	-	-	-
green	-	s_{off} /-	s_{red} /red	s_{red} /red
red	-	s_{off} /-	s_{green} /green	s_{green} /green

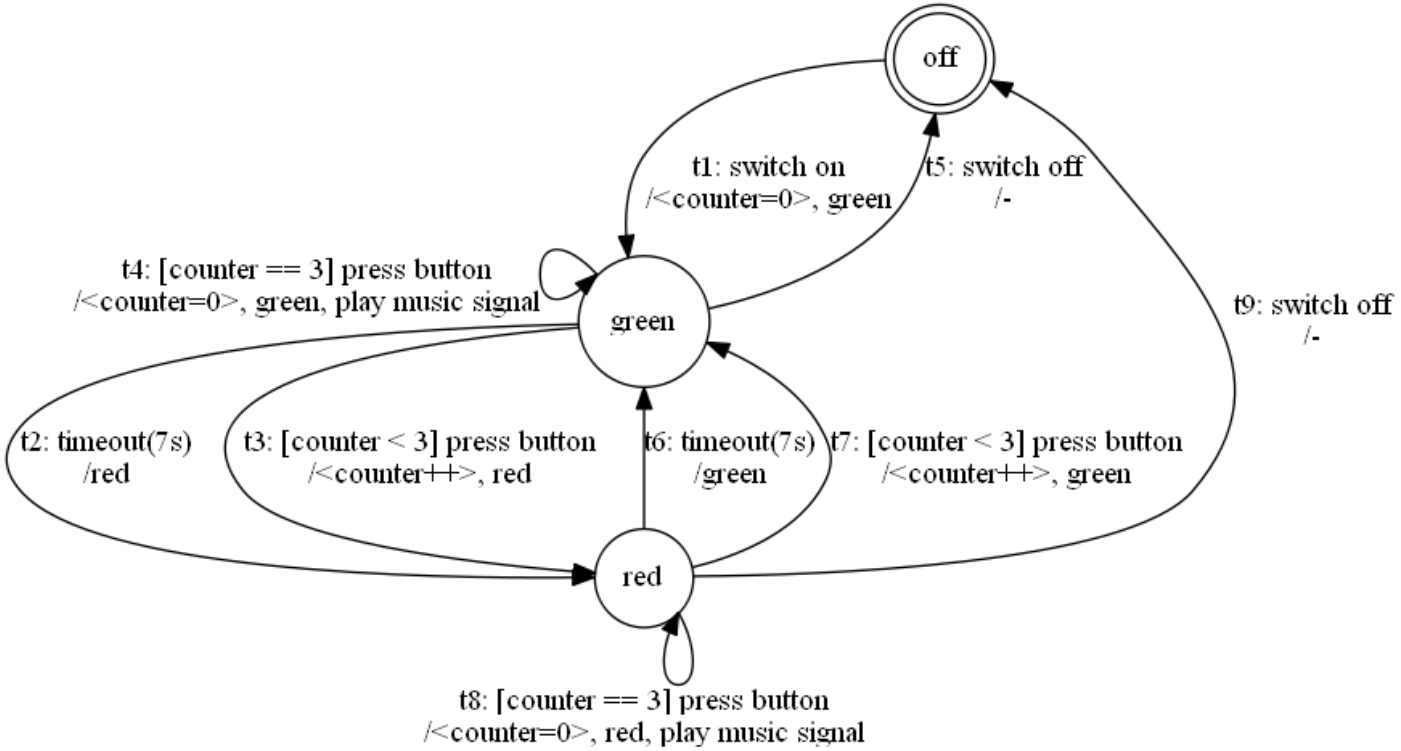
Extend the functionality of the previously discussed battery-operated signal (see spec above) with the following:

- If the button has been pressed *4* times, then the device will play a music signal and it won't change its' state
- The counter of button presses is set to *0* when the music signal ends or if the device is switched on .

EFSMs - solution

Note that the counter is incremented after condition evaluation (as input proceeds output)

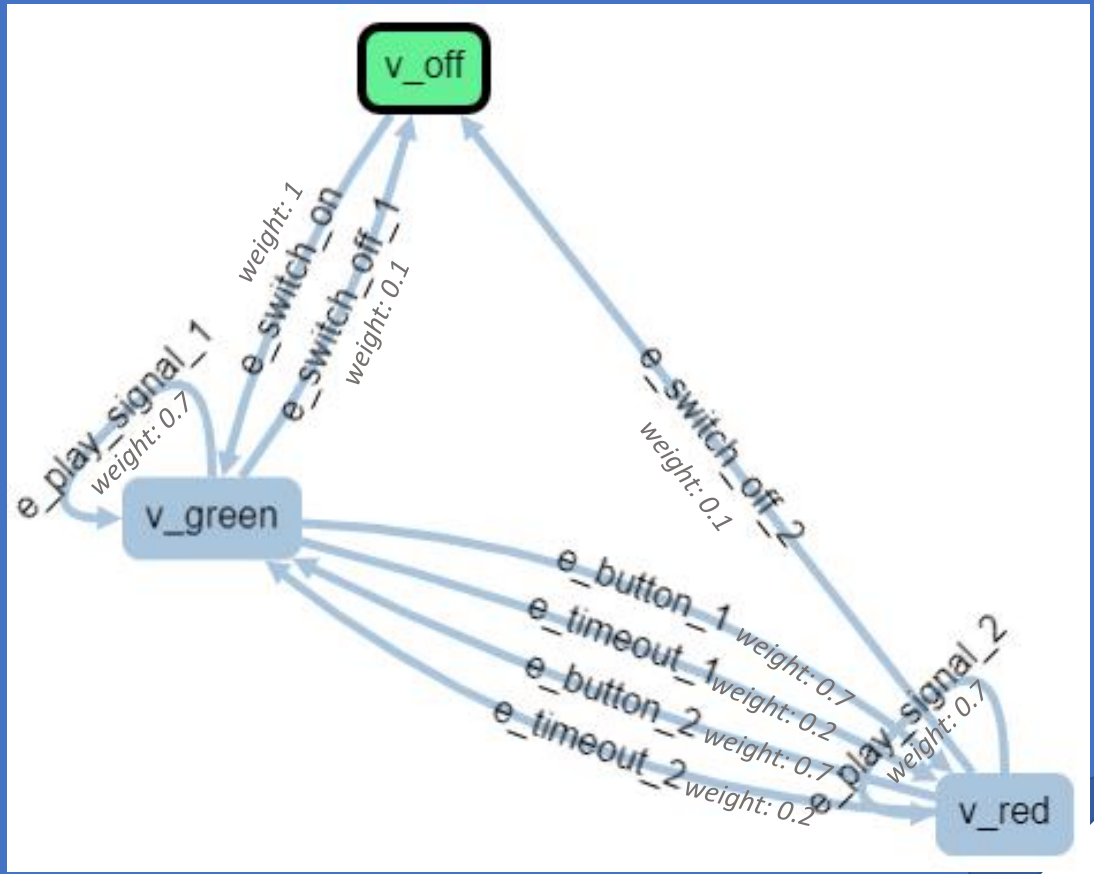
	switch on	switch off	[counter < 3] press button	[counter == 3] press button	wait for 7 secs
off	green /<counter=0>, green	-	-	-	-
green	-	s_{off} /-	s_{red} /<counter++>, red	s_{green} /<counter=0>, green, play signal	s_{red} /red
red	-	s_{off} /-	s_{green} /<counter++>, green	s_{red} /<counter=0>, red, play signal	s_{green} /green



GraphWalker Studio: EFSM for Battery-operated signal

6. Extend the battery-operated signal with guard conditions to play the MÁV music signal after 4th button press instead of changing it's state using yED graph editor
   - Use *counter* variable (initialization, increment, and guard conditions) – see table

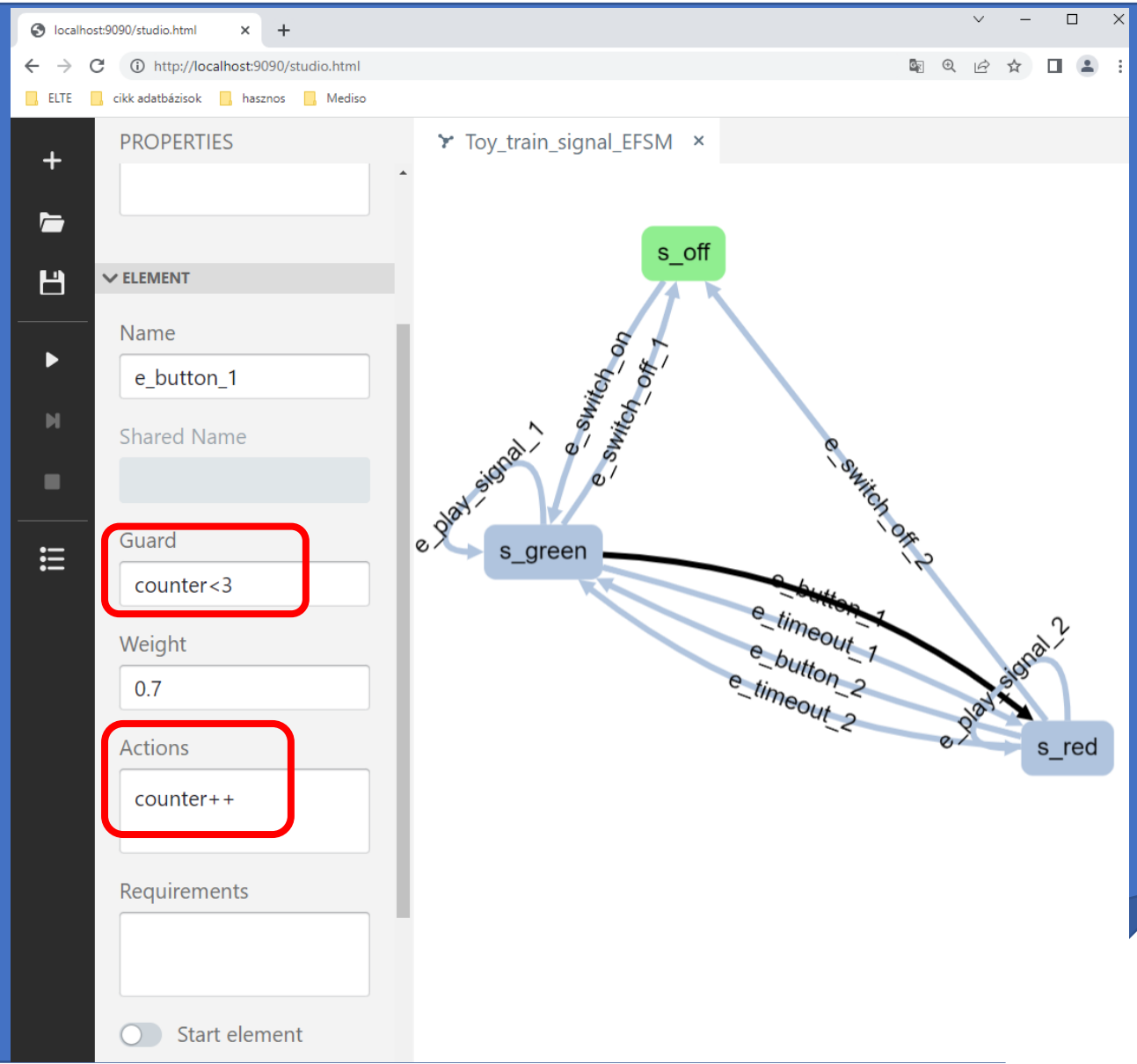
	switch on	switch off	[counter < 3] press button	[counter == 3] press button	wait for 7 secs
off	green /<counter=0>, green	-	off /-	off/-	off /-
green	-	off /-	red /<counter++>, red	green /<counter=0>, green, play signal	red /red
red	-	off /-	green /<counter++>, green	red /<counter=0>, red, play signal	green /green



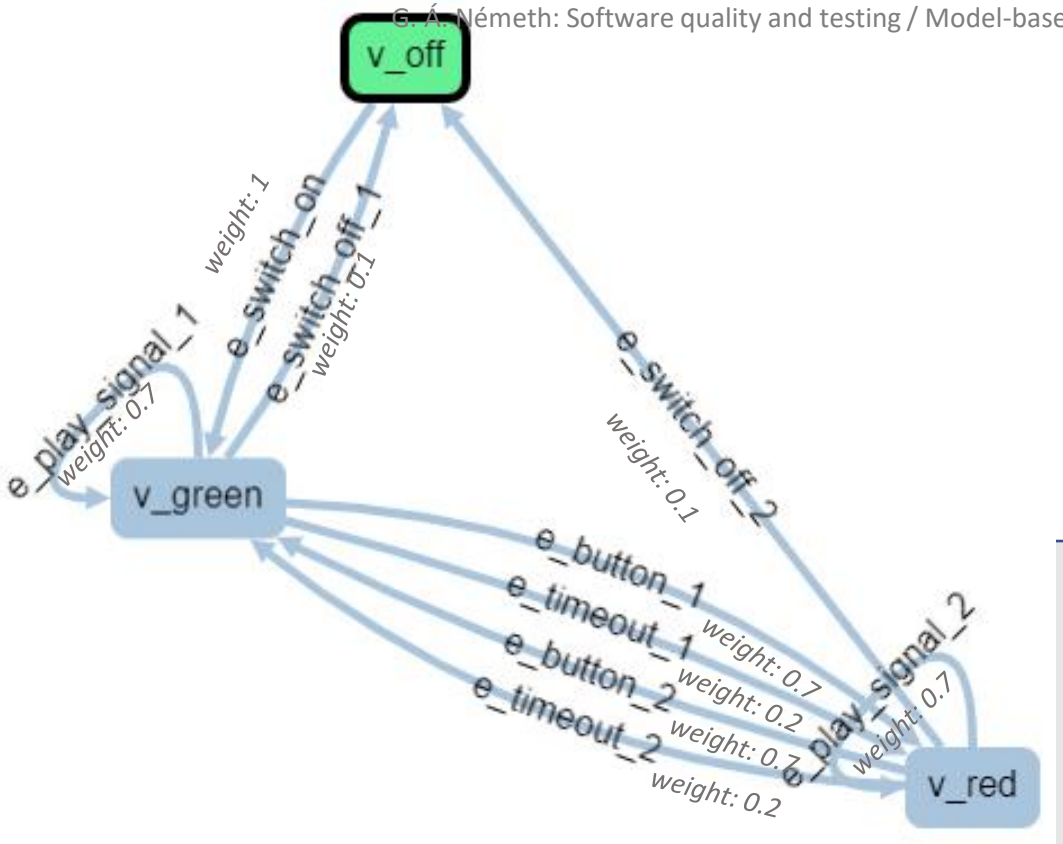
- Add bigger `weights` *(like 0.7)* to button presses and smaller `weights` *(like 0.1)* to switch_off *(otherwise the test would be too long)*
- Use `weighted_random` as generator
- Rerun the tests, what will be the difference?

GraphWalker Studio: EFSM for Battery-operated signal

The syntax of guard conditions and actions over variables



GraphWalker Studio: EFSM for Battery-operated signal



JSON file of the EFSM:


```
{"models":
[
{"name":"Marklin_signal",
                "id":"08414683-68cb-4f68-be4c-670cd27f6cbe",
                "vertices":
                [
                                {"id":"n0","name":"v_green","properties":{"x":20,"y":263.5}},
                                {"id":"n1","name":"v_red","properties":{"x":364,"y":392}},
                                {"id":"n2","name":"v_off","properties":{"x":112,"y":68}}
                ],
                "edges":
                [
                {"id":"e0","name":"e_button_1","guard":"counter<3","actions":["counter++;"],"weight":"0.7","dependency":0,"sourceVertexId":"n0","targetVertexId":"n1"},
                {"id":"e1","name":"e_switch_on","actions":["counter=0;"],"weight":"1","dependency":0,"sourceVertexId":"n2","targetVertexId":"n0"},
                {"id":"e2","name":"e_switch_off_2","actions":[""],"weight":"0.1","dependency":0,"sourceVertexId":"n1","targetVertexId":"n2"},
                {"id":"e3","name":"e_timeout_1","guard":"","actions":[""],"weight":"0.2","dependency":0,"sourceVertexId":"n0","targetVertexId":"n1"},
                                {"id":"e4","name":"e_button_2","guard":"counter<3","actions":["counter++;"],"weight":"0.7","dependency":0,"sourceVertexId":"n1","targetVertexId":"n0"},
                {"id":"e5","name":"e_timeout_2","guard":"","actions":[""],"weight":"0.2","dependency":0,"sourceVertexId":"n1","targetVertexId":"n0"},
                {"id":"e6","name":"e_switch_off_1","actions":[""],"weight":"0.1","dependency":0,"sourceVertexId":"n0","targetVertexId":"n2"},

                {"id":"e7","name":"e_play_signal_1","guard":"counter==3","actions":[""],"properties":{"description":""},"weight":"0.7","dependency":0,"sourceVertexId":"n0","targetVertexId":"n0"},

                {"id":"e8","name":"e_play_signal_2","guard":"counter==3","actions":["counter=0;"],"properties":{"description":""},"weight":"0.7","dependency":0,"sourceVertexId":"n1","targetVertexId"
:"n1"}
                ],
                "generator":"weighted_random(edge_coverage(75))","startElementId":"n2"
}
],
"selectedModelIndex":0,"selectedElementId":"n2"}
```


2 new edges for music signal

Model 》 Test 》 Relax: Import an EFSM model from GraphWalker

- Add the types ("number" or "boolean") of variables manually into the JSON model file (after the "type" keyword for the variable given by it's "id" under the "variables" structure;
  - if the type is not declared for a variable, then MTR assumes it's type is "number".
- Note that the range of these variables can be also set after the "min" and "max" keywords.

```
{
   "models":[
      {
         "name":"Toy_train_signal_EFSM",
         "startElementId":"n2",
         "id":"08414683-68cb-4f68-be4c-
670cd27f6cbe",
         "variables": [
            {
              "id": "counter",
              "type": "number",
              "min": 0,
              "max": 3
            }
          ],
         "vertices":[
            {
               "id":"n0",
               "name":"s_green",
               "properties":{
                  "x":20,
                  "y":263.5
               }
            },
            {
               "id":"n1",
               "name":"s_red",
               "properties":{
                  "x":364,
                  "y":392
               }
            },
            {
               "id":"n2",
               "name":"s_off",
               "properties":{
                  "x":112,
                  "y":68
               }
            }
         ],
         "edges":[
            {
               "id":"e0",
               "name":"e_button_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "guard":"counter<3",
               "actions":["counter++"],
               "input" : "press button",
               "output" : "red",
               "weight": "0.7"
            },
            {
               "id":"e1",
               "name":"e_switch_on",
               "sourceVertexId":"n2",
               "targetVertexId":"n0",
               "actions":["counter=0"],
               "input" : "switch on",
               "output" : "green",
               "weight":"1"
            },
            {
               "id":"e2",
               "name":"e_switch_off_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight":"0.1"
            },
            {
               "id":"e3",
               "name":"e_timeout_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n1",
               "input" : "timeout(7s)",
               "output" : "red",
               "weight":"0.2"
            },
            {
               "id":"e4",
               "name":"e_button_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "guard":"counter<3",
               "actions":["counter++"],
               "input" : "press button",
               "output" : "green",
               "weight":"0.7"
            },
            {
               "id":"e5",
               "name":"e_timeout_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n0",
               "input" : "timeout(7s)",
               "output" : "green",
               "weight":"0.2"
            },
            {
               "id":"e6",
               "name":"e_switch_off_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n2",
               "input" : "switch off",
               "output" : "",
               "weight":"0.1"
            },
            {
               "id":"e7",
               "name":"e_play_signal_1",
               "sourceVertexId":"n0",
               "targetVertexId":"n0",
               "guard":"counter==3",
               "actions":["counter=0;"],
               "input" : "press button",
               "output" : "play music signal",
               "weight":"0.7"
            },
            {
               "id":"e8",
               "name":"e_play_signal_2",
               "sourceVertexId":"n1",
               "targetVertexId":"n1",
               "guard":"counter==3",
               "actions":["counter=0;"],
               "input" : "press button",
               "output" : "play music signal",
               "weight":"0.7"
            }
         ]
      }
   ]
}
```

Note that this model is included in the [Model 》 Test 》 Relax](#) repository:
See `sample_models/introduction/Toy_train_signal_EFSM.json`

EFSM → FSM conversion

- EFSMs (Extended Finite State Machines) can be converted into FSMs
  - All state-variable value combinations of the EFSM → distinct state in the FSM
    - It results in a state explosion problem
  - Many approaches to solve the state explosion problem
    - Generate "collapsed FSM" on-the-fly from EFSM
      - only those variable-state combinations that are used in the EFSM and are reachable from the initial state are generated
    - Decomposition*
      - Functionalities
      - Use cases

* *Note that decomposition may simplify the original problem as restricts interaction between different components (i.e. trade-off between transparency of the model and the coverage of all possible errors / functionality)*

Model 》 Test 》 Relax: EFSM → FSM conversion

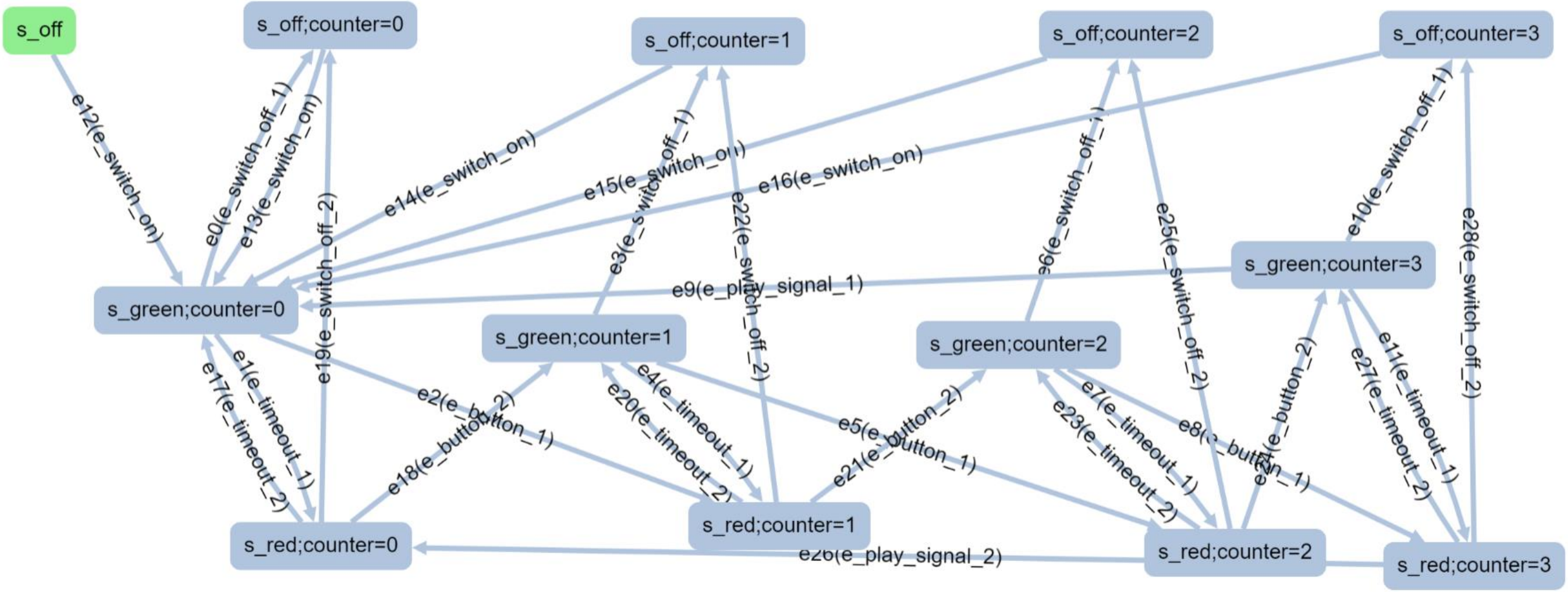
- Convert an EFSM model into an FSM model:

```
./MTR -o conversion -m efsm_to_fsm -f sample_models/introduction/Toy_train_signal_EFSM.json
```

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-JP   ] [warning] No initialization actions defined in JSON model!
[M-FSM  ] [warning] Empty string for output for transition id: e2, name: e_switch_off_2
[M-FSM  ] [warning] Empty string for output for transition id: e6, name: e_switch_off_1
[C-EFSM ] [info] Starting transformation on model: Toy_train_signal_EFSM
[C-EFSM ] [info] Finished computation at 2024-03-19 11:17:24.5486603, elapsed time: 0.001000(user time) seconds
[C-EFSM ] [info] All possible states visited, state count: 13, arc count: 29
[C-EFSM ] [info] finished computation at 2024-03-19 11:17:24.5486603
elapsed time: (real time)0.000969 s
elapsed time (user time)0.001000
[U-CTRL ] [info] Transformation done, output path: converted/Toy_train_signal_EFSM-transformed.json
```

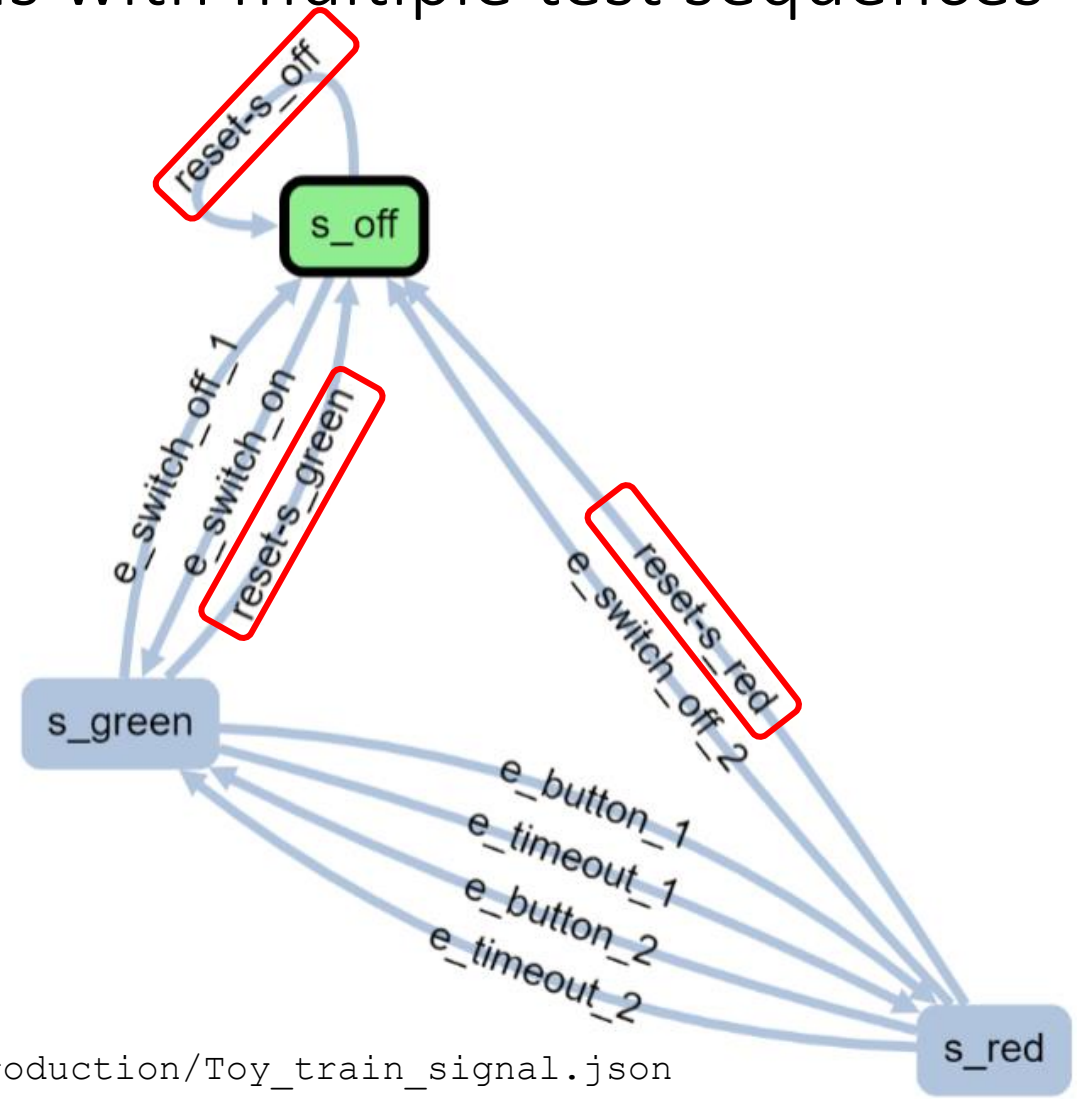
Model 》 Test 》 Relax: EFSM → FSM conversion

- Open the resulting FSM model in GraphWalker Studio *(after rearranging the positions of the states in the canvas)*:



Note that this model is included in OneDrive:
See `sample models used in the course/Toy_train_signal_EFSM_transformed.json`

Investigate FSM test generation algorithms with multiple test sequences



reset message

- takes FSM back to initial state
- Reliable if it is guaranteed to work properly in any implementation

- Add reliable reset to the model:

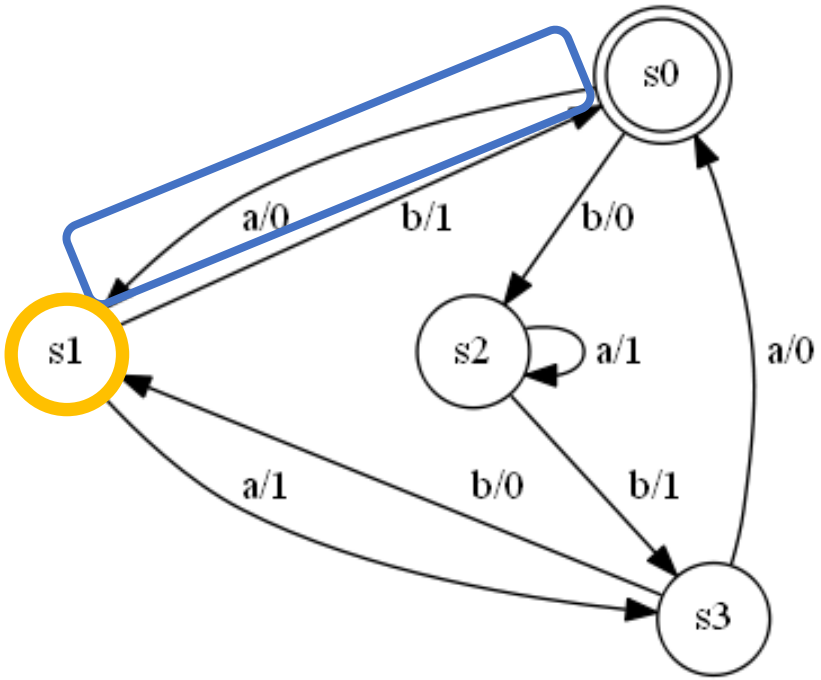
```
./MTR -o conversion -m add_reliable_reset -f sample_models/introduction/Toy_train_signal.json
```

```
D:\MTR\test version\MTR-3-4-0>MTR -o conversion -m add_reliable_reset -f sample_models/introduction/Toy_train_signal.json
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-CTRL ] [info] Reliable reset succesfully enabled for model: sample_models/introduction/Toy_train_signal.json
```

Investigate FSM test generation algorithms with multiple test sequences

0. Reset

1. Preamble
   - To reach a given state

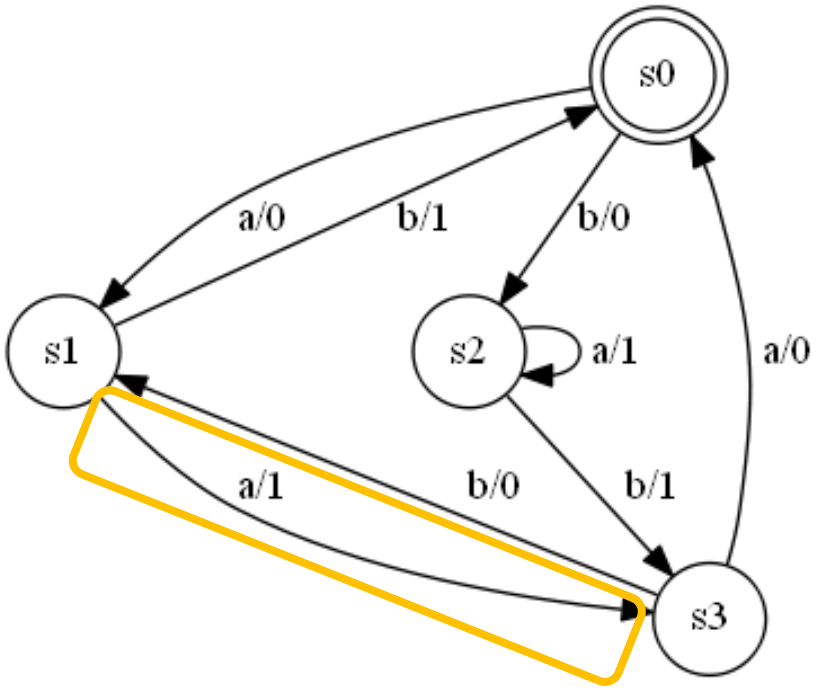


Investigate FSM test generation algorithms with multiple test sequences

0. Reset

1. Preamble
   - To reach a given state

2. Test a given transition
   - To find possible output faults



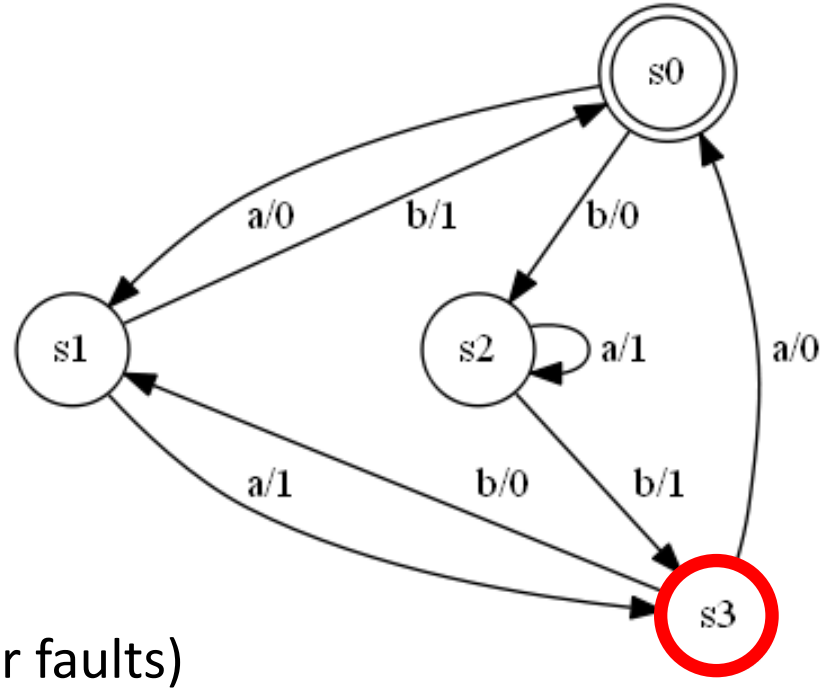
Investigate FSM test generation algorithms with multiple test sequences

0. Reset

1. Preamble
   - To reach a given state

2. Test a given transition
   - To find possible output faults

3. Postamble
   - To verify end state of transition (to find possible transfer faults)



Overall test set consists of test sequences for all states s_x and for all transitions $t_k=(s_x, s_y, i_m\ o_n)$:

reset.reach(s_x). t_k .verify(state s_x)

Investigate FSM test generation algorithms with multiple test sequences

- Apply a test generation algorithm (HSI-method) that contains multiple test sequences:

```
./MTR -o t -m HSI -f sample_models/introduction/Toy_train_signal.json --predefined_path --transition_list
```

Note that the above test generation works only if reset has been added to the model previously

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: Yes (RELIABLE/FLAG)
[U-CTRL ] [info] Running Harmonized State Identifiers test generation, extra states: 0
[TGC-SPG] [warning] The model has a state pair that cannot be separated nor merged!
[TG-HSI ] [info] Constructing auxiliary state-pair graph with all separating sequences...
[TG-HSI ] [info] Creating state cover set...
[TGC-SCS] [info] q(n0): [ switch on ]
[TGC-SCS] [info] q(n1): [ switch on timeout(7s) ]
[TGC-SCS] [info] q(n2): [ ]
[TG-HSI ] [info] Collecting input and output symbols...
[TG-HSI ] [info] Z set:
[TG-HSI ] [info] Z(s_off): z(s_off,s_red): [ ] z(s_off,s_green): [ ]
[TG-HSI ] [info] Z(s_red): z(s_off,s_red): [ ] z(s_red,s_green): [ timeout(7s) ]
[TG-HSI ] [info] Z(s_green): z(s_off,s_green): [ ] z(s_red,s_green): [ timeout(7s) ]
[TG-HSI ] [info] 1st stage: State verification
[TG-HSI ] [info] Verifying state s_red
[TG-HSI ] [info] Verifying state s_green
[TG-HSI ] [info] Verifying state s_off
[TG-HSI ] [info] Inserting and checking the remaining transitions...
[TG-HSI ] [info] 2nd stage: Test remaining transitions
[TG-HSI ] [info] Generating test suite...
[TG-HSI ] [info] The length of the test suite is 21
[TG-HSI ] [info] Finished computation at 2024-03-19 11:54:42.3707784
        elapsed time: (real time) 0.0091482 s
        elapsed time: (user time) 0.010000 s
[TG-TGR ] [info] Test generation summary written: test_summary/hsi_result.csv
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-HSI-extra_states-0-test_suite.json
[TG-TGR ] [info] Writing model with predefined path included to: predefined_path_models/Toy_train_signal_HSI.json
```

Investigate FSM test generation algorithms with multiple test sequences

- Open the generated test sequence in *test_suites* folder (`Toy_train_signal-HSI-extra_states-0-test_suite.json`):


```
{
    "test_suite": {
        "name": "Toy_train_signal",
        "id": "08414683-68cb-4f68-be4c-
670cd27f6cbe",
        "method": "HSI",
        "input_list": [
            "<reset>",
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "timeout(7s)",
            "<reset>",
            "switch on",
            "press button",
            "timeout(7s)",
            "<reset>",
            "switch on",
            "timeout(7s)",
            "press button",
            "timeout(7s)",
            "<reset>",
            "switch on",
            "timeout(7s)",
            "switch off",
            "<reset>",
            "switch on",
            "switch off"
        ],
        "output_list": [
            "",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "green",
            "red",
            "",
            "green",
            "red",
            "",
            "",
            "green",
            ""
        ],
        "transition_id_list": [
            "reset-n2",
            "e1",
            "e3",
            "e5",
            "e3",
            "reset-n1",
            "e1",
            "e0",
            "e5",
            "reset-n0",
            "e1",
            "e3",
            "e4",
            "e3",
            "reset-n1",
            "e1",
            "e3",
            "e2",
            "reset-n2",
            "e1",
            "e6"
        ],
        "transition_name_list": [
            "reset-s_off",
            "e_switch_on",
            "e_timeout_1",
            "e_timeout_2",
            "e_timeout_1",
            "reset-s_red",
            "e_switch_on",
            "e_button_1",
            "e_timeout_2",
            "reset-s_green",
            "e_switch_on",
            "e_timeout_1",
            "e_button_2",
            "e_timeout_1",
            "reset-s_red",
            "e_switch_on",
            "e_timeout_1",
            "e_switch_off_2",
            "reset-s_off",
            "e_switch_on",
            "e_switch_off_1"
        ]
    }
}
```


```
[TG-HSI ] [info] The length of the test suite is 21
```

Investigate FSM test generation algorithms with multiple test sequences

- Apply another test generation algorithm (H-method, improvement of HSI) that contains multiple test sequences:

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --predefined_path --transition_list
```

Note that the above test generation works only reset has been added to the model previously

- Open the generated test sequence in *test_suites* folder (*Toy_train_signal-H-node_order-4-extra_states-0-*


```
{
    "test_suite": {
        "name": "Toy_train_signal",
        "id": "08414683-68cb-4f68-be4c-670cd27f6cbe",
        "method": "H",
        "input_list": [
            "<reset>",
            "switch on",
            "timeout(7s)",
            "switch off",
            "<reset>",
            "switch on",
            "timeout(7s)",
            "timeout(7s)",
            "<reset>",
            "switch on",
            "timeout(7s)",
            "press button",
            "<reset>",
            "switch on",
            "switch off",
            "<reset>",
            "switch on",
            "press button"
        ],
        "output_list": [
            "",
            "green",
            "red",
            "",
            "",
            "green",
            "red",
            "green",
            "",
            "green",
            "red",
            "green",
            "",
            "green",
            "",
            "",
            "green",
            "red"
        ],
        "transition_id_list": [
            "reset-n2",
            "e1",
            "e3",
            "e2",
            "reset-n2",
            "e1",
            "e3",
            "e5",
            "reset-n0",
            "e1",
            "e3",
            "e4",
            "reset-n0",
            "e1",
            "e6",
            "reset-n2",
            "e1",
            "e0"
        ],
        "transition_name_list": [
            "reset-s_off",
            "e_switch_on",
            "e_timeout_1",
            "e_switch_off_2",
            "reset-s_off",
            "e_switch_on",
            "e_timeout_1",
            "e_timeout_2",
            "reset-s_green",
            "e_switch_on",
            "e_timeout_1",
            "e_button_2",
            "reset-s_green",
            "e_switch_on",
            "e_switch_off_1",
            "reset-s_off",
            "e_switch_on",
            "e_button_1"
        ]
    }
}
```


```
[TG-H    ] [info] The length of the test suite is 18
```

Part 3

- Tradeoff between test generation complexity and fault detection capability
- Handle unreliable reset symbols
- Compare the complexity of test generation algorithms
- Some notes related to adaptation code
- Conclude Model-based testing

Investigate FSM test generation algorithms with multiple test sequences

Tradeoff between test generation complexity and fault detection capability:

- Apply the previous test generation algorithm (H-method) again and again with different settings for possible extra states in implementation:

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json
```

```
[TG-H   ] [info] The length of the test suite is 18
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=1
```

```
[TG-H   ] [info] The length of the test suite is 51
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=2
```

```
[TG-H   ] [info] The length of the test suite is 152
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=3
```

```
[TG-H   ] [info] The length of the test suite is 431
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=4
```

```
[TG-H   ] [info] The length of the test suite is 1198
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=5
```

```
[TG-H   ] [info] The length of the test suite is 3271
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=10
```

```
[TG-H   ] [info] The length of the test suite is 423680
elapsed time: (user time) 22.099000 s
```

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --extra_states=15
```

You won't have time to wait for the results… ☺

There is always s an opportunity to increase the coverage of tests, but it comes with a price (increase in test generation and test execution complexity)

Investigate FSM test generation algorithms with multiple test sequences

- What can we do if a reset is unreliable?
  - We must test all reset transitions first

    (see [Test unreliable reset symbols](#) of the framework)

- Consider, that the given model has unreliable reset capability (add unreliable reset with the below command):

```
./MTR -o conversion -m add_unreliable_reset -f sample_models/introduction/Toy_train_signal.json
```

```
D:\MTR\test version\MTR-3-4-0>MTR -o conversion -m add_unreliable_reset -f sample_models/introduction/Toy_train_signal.json
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-CTRL ] [info] Unreliable reset succesfully enabled for model: sample_models/introduction/Toy_train_signal.json
```

- Run the previous test generation again:

```
./MTR -o t -m H -f sample_models/introduction/Toy_train_signal.json --predefined_path --transition_list
```

- What will be the difference?

```
[U-CTRL ] [info] Version: 3.4.0 R4: Belted kingfisher, profile: DEFAULT, verbosity: 3, debug mode: off
[U-LMI  ] [info] Model Name: Toy_train_signal, Type: FSM, Reset: Yes (UNRELIABLE/FLAG)
[U-CTRL ] [info] Running H-method test generation, node order: 4, extra states: 0
[TG-TGR ] [info] The reset symbols of the given model are unreliable. Generating Reset Check Sequence,
that should be applied before use the test suite of this test generation algorithm.
[U-TW   ] [info] Test suite written: test_suites/Toy_train_signal-RCS-test_suite.json
```

Other test generation algorithms:

N-Switch coverage (N-SC):

- Covers all topologically possible, consecutive *N+1* transitions
  - 0 switch coverage: Transition-Tour method
- Complexity dramatically increases if *N* or the size of the specification increases!

Again, there is always s an opportunity to increase the coverage of tests, but it comes with a price (increase in test generation and test execution complexity)

1-switch coverage: `./MTR -m NS -f sample_models/introduction/Toy_train_signal.json`

```
[TG-1-SC] [info] Result:
        length of test sequence: 26,
```

2-switch coverage: `./MTR -m NS --ns_value=2 -f sample_models/introduction/Toy_train_signal.json`

```
[TG-2-SC] [info] Result:
        length of test sequence: 64,
```

3-switch coverage: `./MTR -m NS --ns_value=3 -f sample_models/introduction/Toy_train_signal.json`

```
[TG-3-SC] [info] Result:
        length of test sequence: 168,
```

4-switch coverage: `./MTR -m NS --ns_value=4 -f sample_models/introduction/Toy_train_signal.json`

```
[TG-4-SC] [info] Result:
        length of test sequence: 402,
```

5-switch coverage: `./MTR -m NS --ns_value=5 -f sample_models/introduction/Toy_train_signal.json`

```
[TG-5-SC] [info] Result:
        length of test sequence: 1042,
```

Model 》 Test 》 Relax: investigate different test generation algorithms

To use the simulation script of Model 》 Test 》 Relax to be able to compare different test generation algorithms, you must install the following:

- **Python 3:**
  - **Download from here** for Windows
  - For **Linux** it can be downloaded with a package manager: `sudo apt install python3`
- Some python packages *(can be installed with pip using the following commands)*:

```
pip install pandas
pip install matplotlib
pip install plotly
pip install kaleido
pip install openpyxl
```

Model 》 Test 》 Relax: investigate different test generation algorithms

- Simulation script - Batch job can be executed:
  1. Model generation:
     - Run random model generation - params:
       - Number of states: fix / given range with given increments
       - Density: fix / given range with given increments
       - Number of output symbols: fix / given range with given increments
  2. Test generation:
     - Apply the list of test generation algorithms with given parameters for a given list of models
     - The results will be collected in a csv and a summary xls file
     - In case of a server outage one can continue test generation
  3. Fault coverage investigations:
     - Apply given number of faults to the models and check if the tests created by given test generation algorithms can discover them
  4. Draw diagram from the csv file

Model 》 Test 》 Relax: investigate different test generation algorithms

Commands *(from `./.../MTR-x-y-z/simulations/scripts/` folder)* for [simulating script for batch execution](#):

1. Generation of random models

```
python3 run_sims.py --model_gen s 10 100 5 d 5 o 10 spanning reliable
```

Rename folders:

- Rename folder (in linux): `mv models_s_10_100_5_d_5_o_10_st_ps_r_* scenario_1`
- Rename folder (in windows): `Rename models_s_10_100_5_d_5_o_10_st_ps_r_* my_scenario_models`

*(Note that the * wildcard character may not work for windows. In this case, instead of * the user can press the TAB to fill in the appropriate hash automatically.)*

2. Test generation for generated models with selected test generation algorithms:

```
python3 run_sims.py --batch_exec my_models/ ATS TT H --profile RESEARCH
```

The results are collected on separate csv files (distinct file for each test generation algorithm) an a common summary.xslx:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD
1		model_nam	del_ty	ate_cou	nsition_co	density	_symbol	_symbol	has_reset	et_reliabi	ble_reset_	eneration	ersion_me	letely_spe	method	rt_timesta	sh_timesta	uration_re	uration_us	equence_	en_mem_	l_mem_us	at_path_s	tsp_path_	at_path_s	tsp_path_s	at_path_s	tsp_path_	_at_paths	tsp_paths
2	0	model_s_	FSM	10	50	5	8	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0039352	0,004000	167	0	0	60	11	20	17	48	11	0	0
3	1	model_s_	FSM	15	75	5	9	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0090291	0,009000	297	0	0	97	15	40	29	94	22	0	0
4	2	model_s_	FSM	20	100	5	8	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0162654	0,016000	422	0	0	137	23	52	43	139	28	0	0
5	3	model_s_	FSM	25	125	5	9	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0212549	0,022000	494	0	0	175	30	55	43	156	35	0	0
6	4	model_s_	FSM	30	150	5	9	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0442384	0,044000	688	0	0	217	38	79	61	251	42	0	0
7	5	model_s_	FSM	35	175	5	9	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0578076	0,057000	771	0	0	263	43	93	75	245	52	0	0
8	6	model_s_	FSM	40	200	5	11	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,0621538	0,062000	858	0	0	280	48	102	77	274	77	0	0
9	7	model_s_	FSM	45	225	5	11	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,114951	0,115000	1122	0	0	338	50	86	68	508	72	0	0
10	8	model_s_	FSM	50	250	5	10	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,122981	0,123000	1164	0	0	394	66	144	118	368	74	0	0
11	9	model_s_	FSM	55	275	5	10	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,158161	0,158000	1247	0	0	389	70	154	136	420	78	0	0
12	10	model_s_	FSM	60	300	5	8	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,173978	0,174000	1419	0	0	437	72	206	164	444	96	0	0
13	11	model_s_	FSM	65	325	5	13	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,226688	0,226000	1434	0	0	438	79	212	166	443	96	0	0
14	12	model_s_	FSM	70	350	5	10	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,261979	0,262000	1654	0	0	530	86	215	164	554	105	0	0
15	13	model_s_	FSM	75	375	5	12	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,405037	0,405000	1689	0	0	506	92	235	194	543	119	0	0
16	14	model_s_	FSM	80	400	5	9	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,284356	0,284000	1770	0	0	548	96	241	184	587	114	0	0
17	15	model_s_	FSM	85	425	5	11	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,461903	0,462000	2098	0	0	636	105	261	199	751	146	0	0
18	16	model_s_	FSM	90	450	5	10	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	1,31446	1,315000	2015	0	0	634	109	299	216	631	126	0	0
19	17	model_s_	FSM	95	475	5	12	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,566879	0,567000	2140	0	0	665	122	292	212	706	143	0	0
20	18	model_s_	FSM	100	500	5	11	10	IGAZ	RELIABLE	FLAG	SPANNIN	NOT_CON	HAMIS	ATS	2024-03-1	2024-03-1	0,611596	0,612000	2573	0	0	720	126	371	304	881	171	0	0

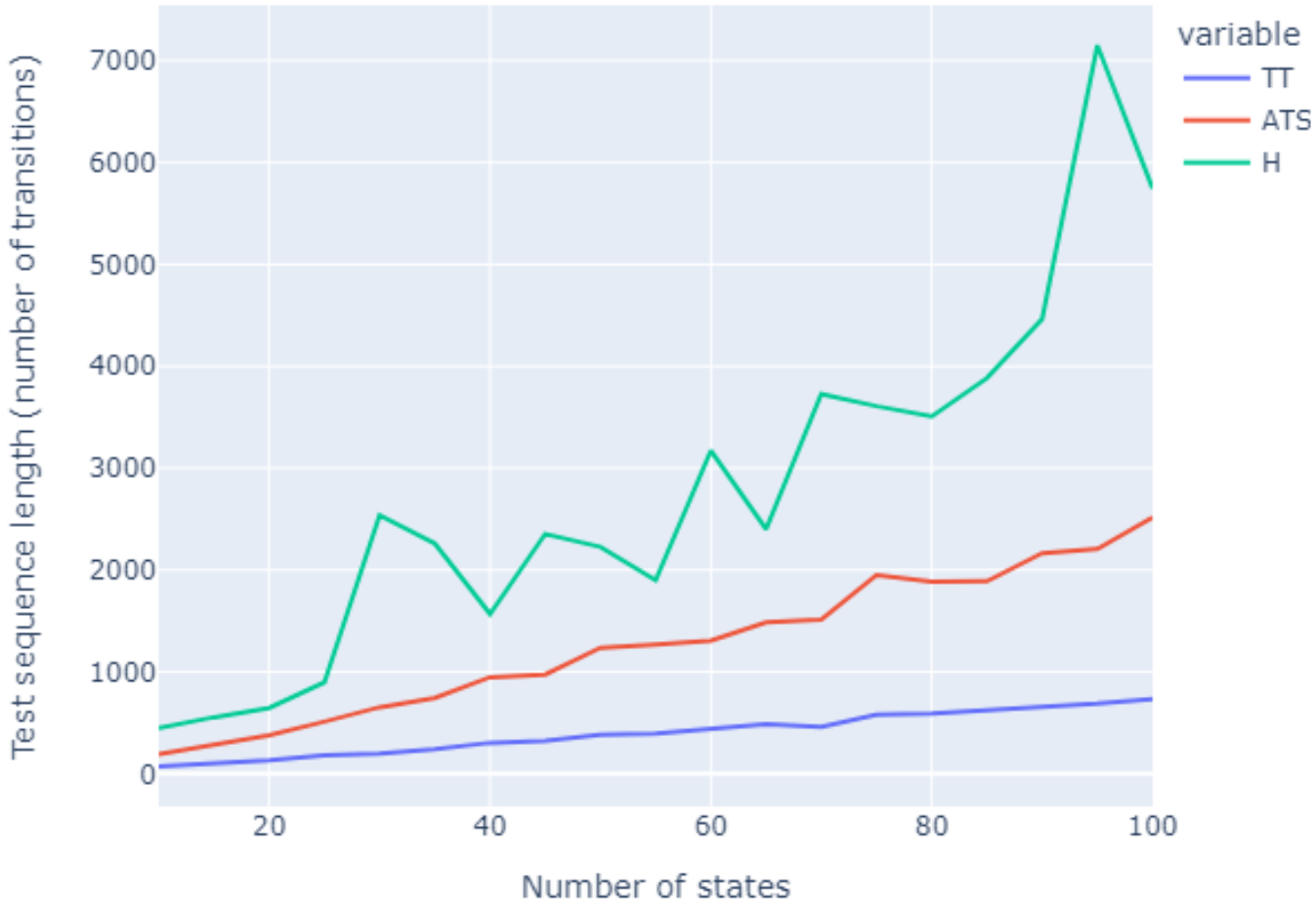
ATS | TT | H

Model 》 Test 》 Relax: investigate different test generation algorithms

Commands *(from `./.../MTR-x-y-z/simulations/scripts/` folder)* for [simulating script for batch execution](#):

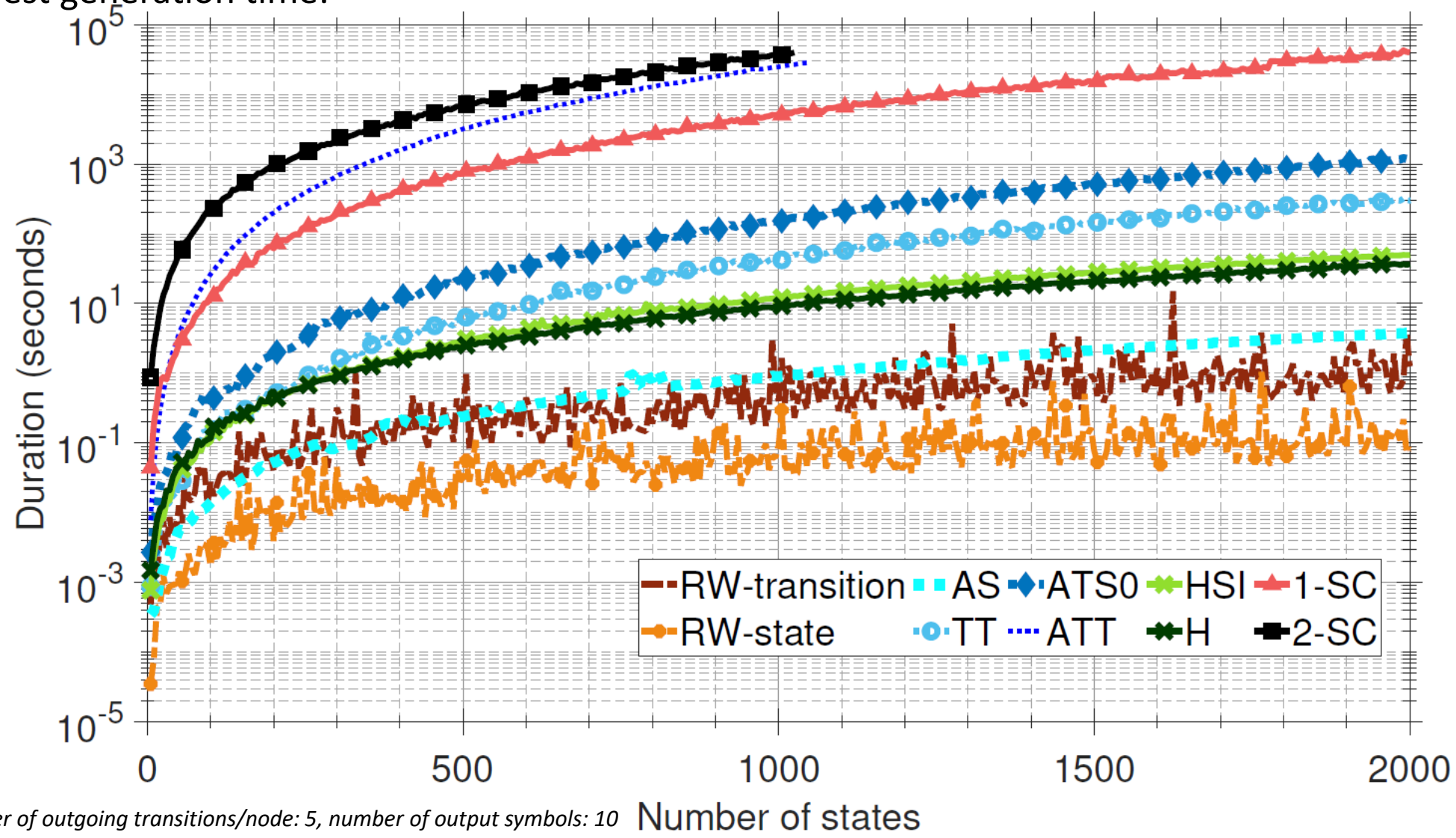
3. Draw diagrams from results *(assuming that the results of test executions are saved into* `test_summary` *subfolder)*:

```
python3 run_sims.py --plotly state_count test_sequence_length test_summary/ats_result.csv test_summary/tt_result.csv test_summary/h_result.csv
```



Complexity of different test generation algorithms

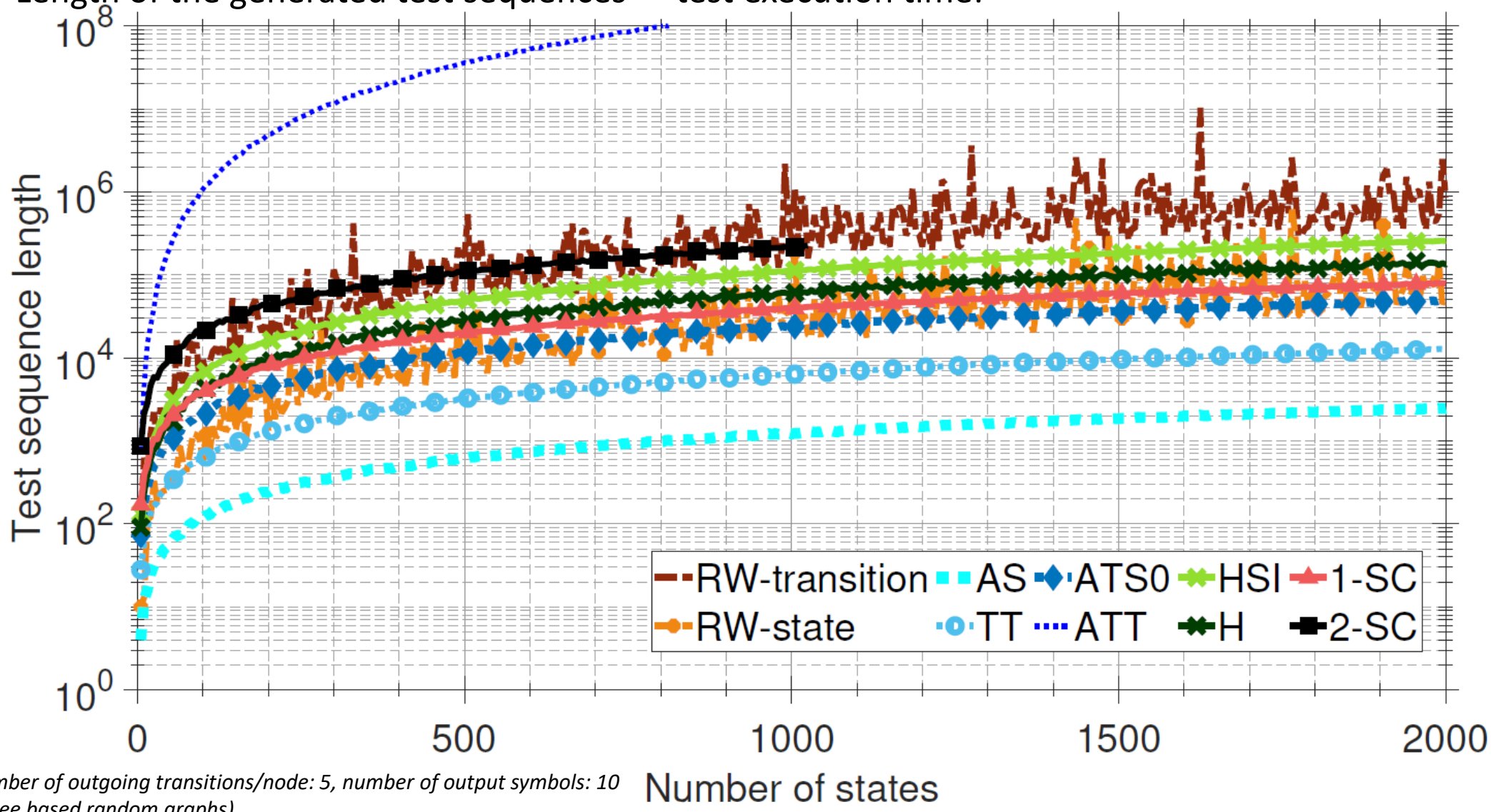
- Test generation time:



Average number of outgoing transitions/node: 5, number of output symbols: 10
(Spanning tree based random graphs)

Complexity of different test generation algorithms

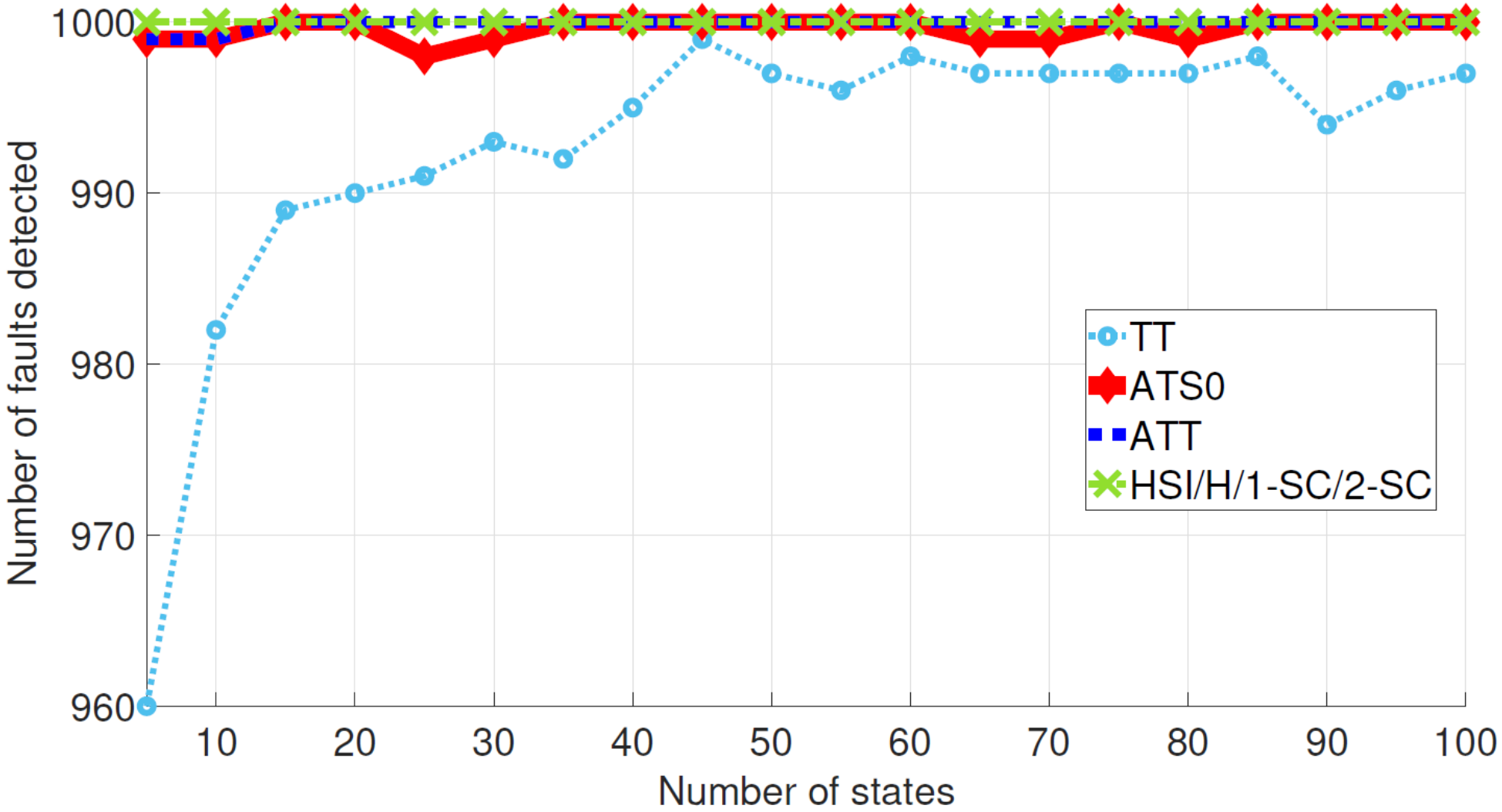
- Length of the generated test sequences → test execution time:



Average number of outgoing transitions/node: 5, number of output symbols: 10
(Spanning tree based random graphs)

Fault coverage of different generated tests

- Fault coverage investigation with randomly injected transfer faults:



Average number of outgoing transitions/node: 5, number of output symbols: 10
(Spanning tree based random graphs)

Adaption code

- Most of the MBT tool generate test code only for **symbolic execution**, i.e. to run on implementation model rather than on actual implementation*
  - **Adaptation code** is required which "adapt" the MBT generated test case to the SUT to be executable
    - i.e. adaptation code (glue code) implement model elements:
      - Transitions of the FSM
    - The used approach depending on the tested application and its interfaces (GUI, API, CLI…etc)

* In some cases – if additional information exist in the specification model about how to connect this model to the SUT, and connections to other tools are provided as well – the adaptation code can be partially or completely generated. But in this case the adaptation of test cases will be restricted to some predefined problem domains based on the applied MBT tool.

API: application programming interface *CLI: Command Line Interface* *GUI: Graphical User Interface* *SUT: System Under Test*

Adaptation code (glue code) examples

GUI testing of a Web application with Selenium:

```
//Adaptation code that simulates the clicking on an element
public static void MakeClick(ChromeDriver driver, int i, string from, string input, string output)
{
    try
    {
        if (input == "press Previous")
        {
           IWebElement pressNext = driver.FindElement(By.XPath("/html/body/div/main/a[2]"));
           pressNext.Click();
        }
        else if(input == "select a category")
        {
           Functions.GoToCategoriesPage(driver);
        }
        IWebElement nextAction = driver.FindElement(By.XPath(Dictionaries.xpaths[input]));
        nextAction.Click();
    }
    catch (Exception e)
    {
        throw new Exception("ERROR: Could not find element by XPath on source page!" +
        "\n" + "test No.: " + i.ToString() + ", test from: " + from + ", test input: " +
        input + ", test output: " + output);
    }
}
```

Testing through HTTP *(Docker)*:

```
//Corresponding model element:
input "start container - successful - not running" {
guard {
return container_state in [CONTAINER_STATE_CREATED, CONTAINER_STATE_EXITED]
}
body { container_state = CONTAINER_STATE_RUNNING }
//Adaptation code for "Start container" transition of the model:
adapter {
res = client.post("/containers/"
+ TEST_CONTAINER_NAME + "/start")
assert res.status_code == 204, "Status code 204 expected"
}
```

- Sample test projects with their adaptation code are provided in Model 》 Test 》 Relax

Testing through an API *(Telegram API)*:

```
//Create connection between model elements and adaptation code:
var tests = new Dictionary<string, Func<Task>> {
    ...
    {"create Chat", CreateChatTest},
...
//Adaptation code for "create Chat" transition of the model:
async Task CreateChatTest()
{
    var contacts = await client.Contacts_GetContacts();
    user = contacts.users[contacts.contacts.First().user_id];

    var update = await client.Messages_CreateChat(new InputUserBase[]
    {
        new InputUserSelf(),
        user
    }, "Test");
    CustomAssert.Assert(update.Chats.Values.FirstOrDefault(x => x.Title
    == "Test") != null, nameof(CreateChatTest));

    chatId = update.Chats.Values.First(x => x.Title == "Test").ID;
}
```

Testing through CLI *(GitHub CLI)*:

```
//Adaptation code for "Open New PR" transition of the model:
def OpenNewPR(self):
    self.prid = self.__get_next_pr_id()
    branch_name = git_new_branch(self.prid)
    sh.gh.pr.create("--fill", "--head",
                    branch_name,
                    _env=get_env("github-fmbt.token"))
    return self.prid
```

Test code in Java → C++ code adapted with a dll file, JNA *(CNC Controller)*:

```
@Override
public void e_start()
{
  step();
}

private void step()
{
  dll.setMotorMoverTarget(ptr, x_on ? 1 : 0, y_on ? 1 : 0, z_on ? 1 : 0, 1, false);
}
```

API: application programming interface
CLI: Command Line Interface
DLL: Dynamic Link Library
GUI: Graphical User Interface
HTTP: HyperText Transfer Protocol
JNA: Java Native Access

MBT: Advantages

- Modeling process
  - Close communication with stakeholders
    - Graphical description helps

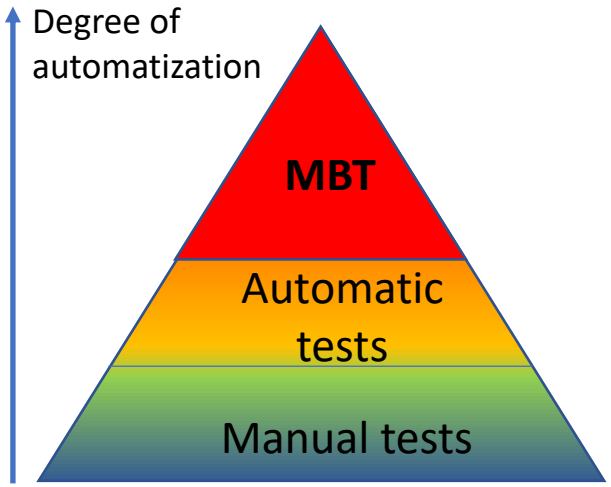
    → common understanding of the problem → identify misunderstandings
  - For engineers models are more understandable than test cases
  - Helps to avoid defects* at early stages

    ** incomplete, inconsistent and unclear requirements*
  - Test generation and analyzation is possible before any real system exists
- Traceability between use cases and tests
- Automated test generation
  - Reduces or eliminates the time of test design
  - Help to reduce maintenance costs when requirements change
  - Flexible: test generation criteria can be changed easily according test requirements

MBT: Disadvantages

- High initial cost
  - Requires different thinking
    - Modeling and adaptation code → More qualified testers are required
- Models can also contain bugs
  - Attention must be paid for this → Model creation is an iterative process
- Hard to add to existing toolchains

Summary of MBT



- Next level of test automatization:
  - Automated test design: model → generated test
- Good at defect prevention, easy maintenance through the evolving of SUT
- Can handle complex systems with hierarchical models
- Requires different type of thinking, high initial cost